第8章 图

提纲

CONTENTS

8.1 图的基本概念

8.2 图的存储结构

8.3 图的遍历

8.3 图遍历算法的应用

8.5 生成树和最小生成树

8.6 最短路径

8.7 拓扑排序

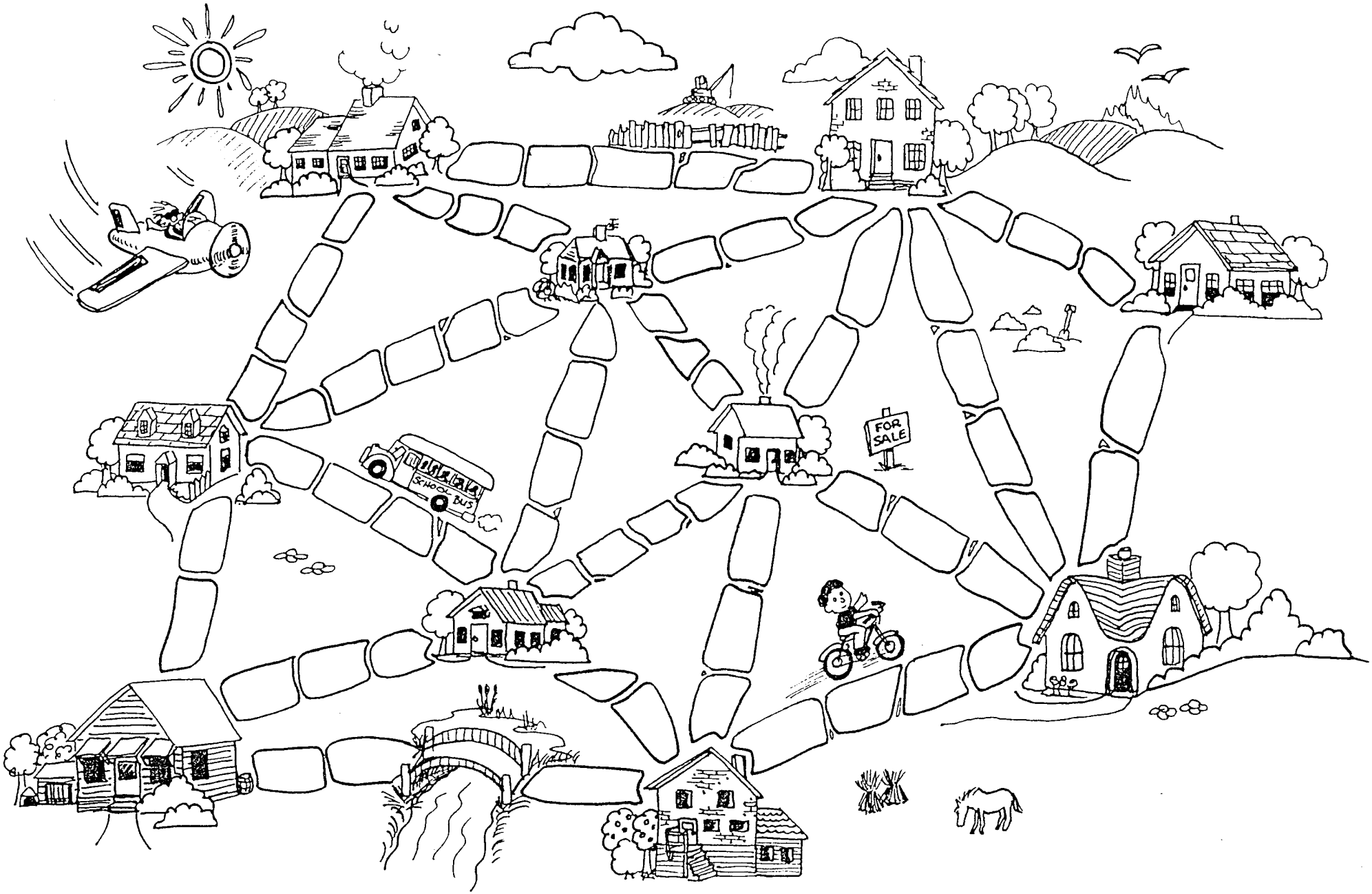
8.8 AOE网与关键路径

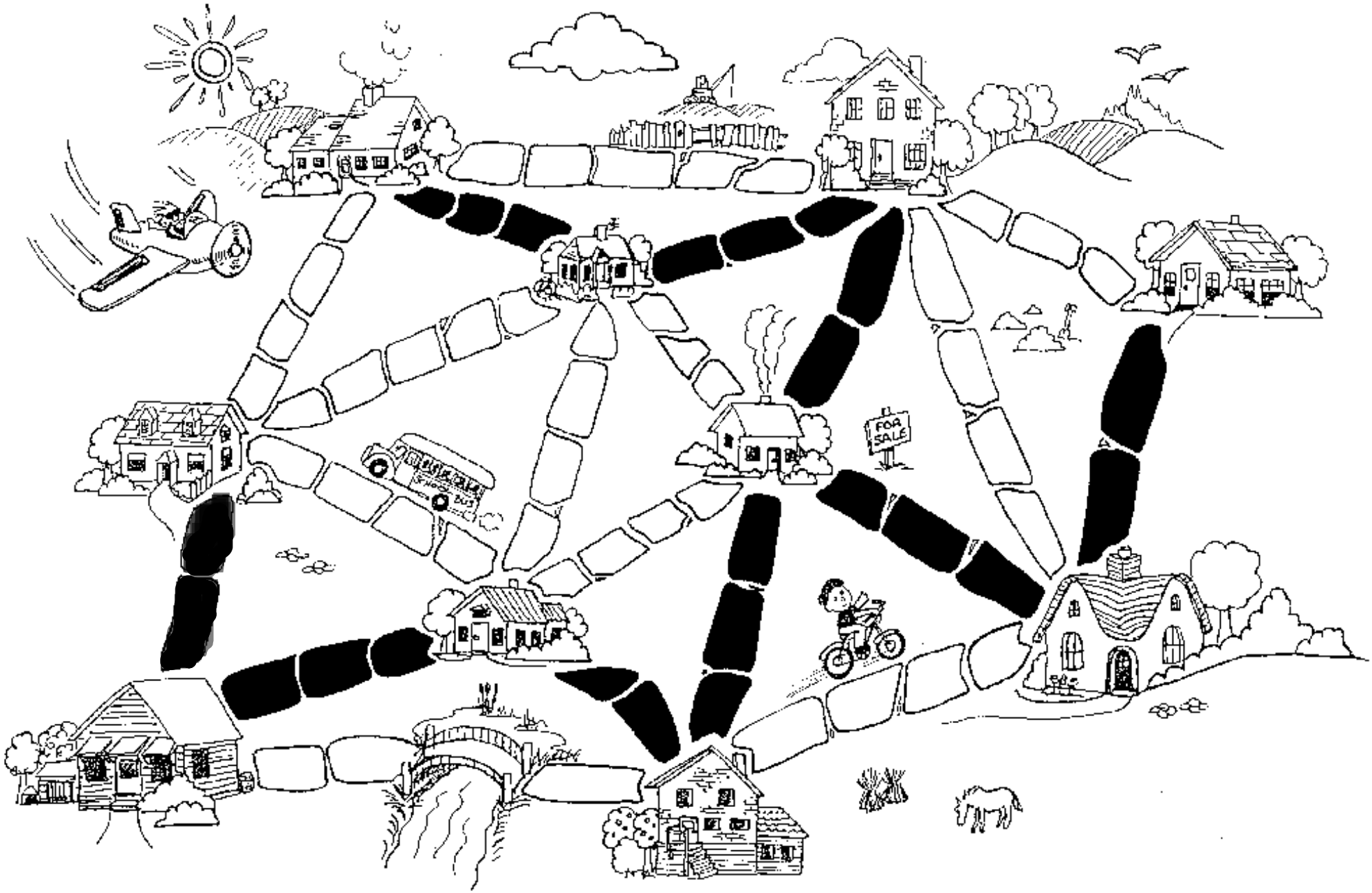
8.5 生成树和最小生成树

8.5.1 生成树和最小生成树的概念

1．什么是生成树

- 一个有n个顶点的连通图的生成树是一个极小连通子图。
- 生成树含有图中全部顶点，但只包含构成一棵树的$n-1$条边。
- 如果在一棵生成树上添加一条边，必定构成一个环：因为这条边使得它依附的那两个顶点之间有了第二条路径。


SCHOOL BUS
FOR SALE

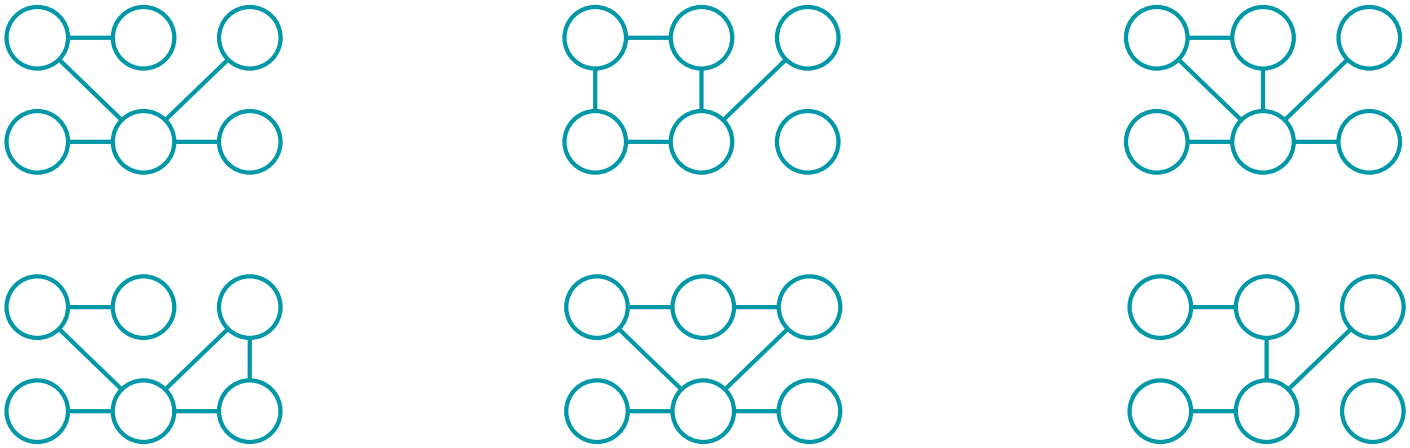

FOR SALE

8.5 生成树和最小生成树

8.5.1 生成树和最小生成树的概念

1. 什么是生成树

下图哪些是生成树？

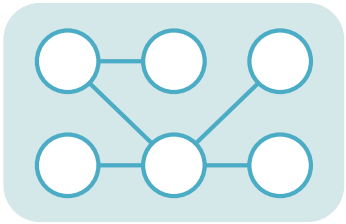


8.5 生成树和最小生成树

8.5.1 生成树和最小生成树的概念

1. 什么是生成树

下图哪些是生成树？



不连通

不是树

不是树

不是树

不连通

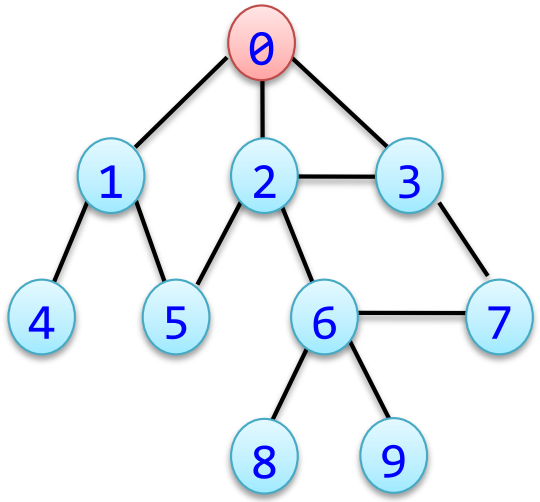
2．连通图的生成树和非连通图的生成森林

- 连通图：仅需调用遍历过程（DFS或BFS）一次，从图中任一顶点出发，便可以遍历图中的各个顶点。遍历中搜索边<v，w>时，若顶点w首次访问（该边也是首次搜索到），则该边是一条树边，所有树边构成一棵生成树。
- 非连通图：需对每个连通分量调用一次遍历过程，所有连通分量对应的生成树构成整个非连通图的生成森林。

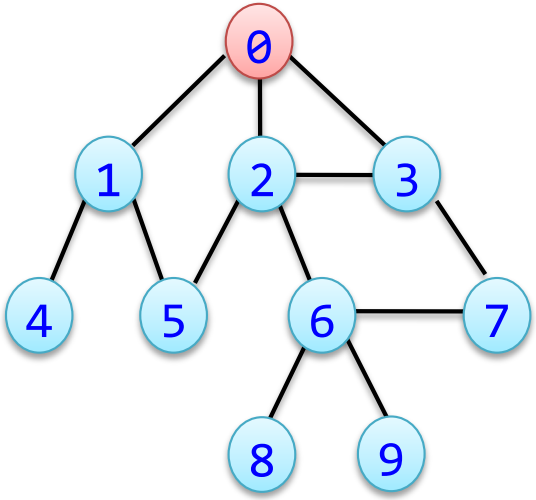
3．由两种遍历方法产生的生成树

- 由深度优先遍历得到的生成树称为深度优先生成树。
- 由广度优先遍历得到的生成树称为广度优先生成树。
- 无论哪种生成树，都是由相应遍历中首次搜索的边构成的。

【例8.11】对于如下的无向图，画出其邻接表存储结构，并在该邻接表中，以顶点0为根，画出图G的深度优先生成树和广度优先生成树。



邻接表中单链表按顶点编号递增排列！



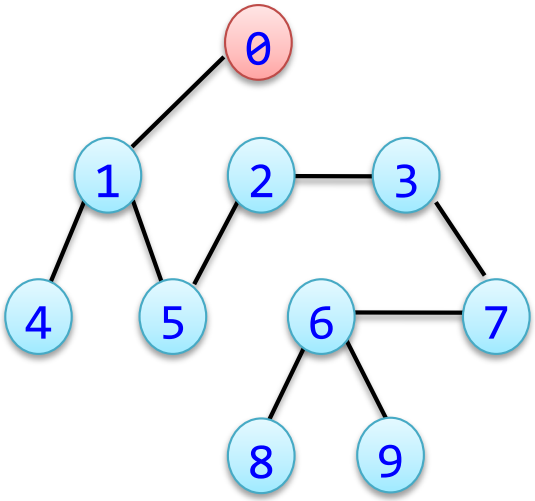
DFS(0)

0 → 1 → 4

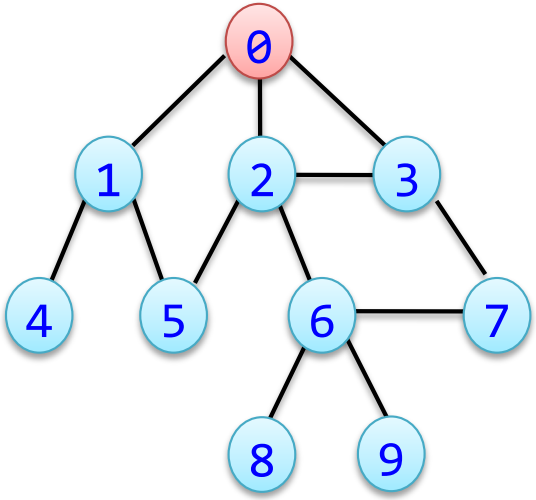
5 → 2 → 3 → 7 → 6 → 8

9

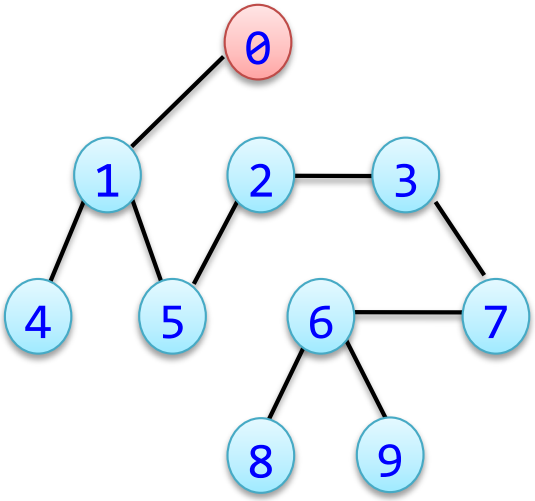
DFS树



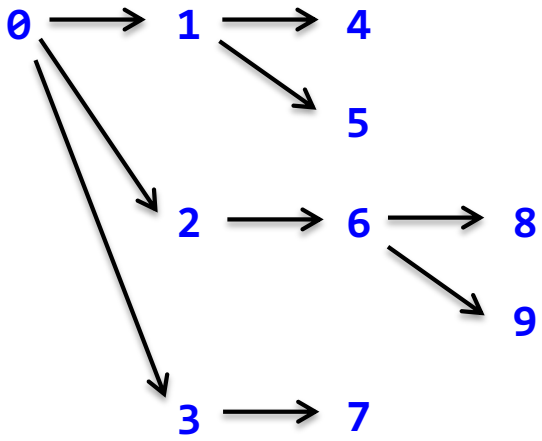
邻接表中单链表按顶点编号递增排列！



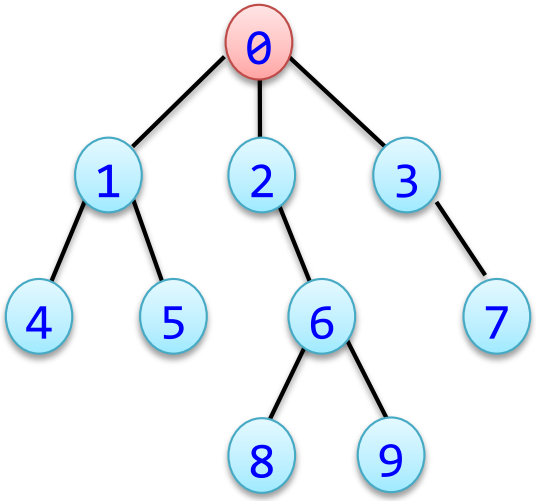
DFS树



BFS(0)



BFS树



4．什么是最小生成树

- 一个带权连通图G（假定每条边上的权值均大于零）可能有多棵生成树．
- 每棵生成树中所有边上的权值之和可能不同．
- 其中边上的权值之和最小的生成树称为图的最小生成树。

最小生成树算法应用场景

- 通信与物理网络设计：
  - 通信网络建设：光缆、电话线网...
  - 电力与管道传输： 输电网、自来水管道、天然气管道...
  - 交通规划
- 电路设计 (VLSI 布局)
- 数据聚类与机器学习：单链聚类算法（Single-linkage Clustering）
- 近似算法的基石：一些 NP-Hard 问题（如旅行商问题 TSP）

8.5.2 普里姆算法

1．普里姆算法过程

普里姆（Prim）算法是一种构造性算法。假设G=(V，E)是一个具有*n*个顶点的带权连通图，T=(U，TE)是G的最小生成树，其中U是T的顶点集，TE是T的边集，则由G构造从起始点*v*出发的最小生成树T的步骤如下：

（1）初始化U={*v*}。以*v*到其他顶点的所有边为候选边。

（2）重复以下步骤*n*-1次，使得其他*n*-1个顶点被加入到U中：

① 从候选边中挑选权值最小的边加入TE（所有候选边一定是连接两个顶点集U和V-U的边），设该边在V-U中的顶点是*k*，将顶点*k*加入U中。

② 考察当前V-U中的所有顶点*j*，修改候选边：若(*k*，*j*)的权值小于原来和顶点*j*关联的候选边，则用(*k*，*j*)取代后者作为候选边。

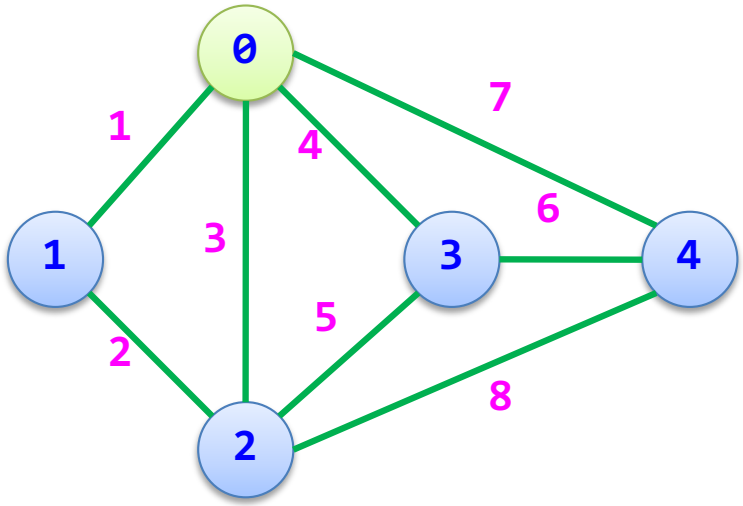
迭代思路

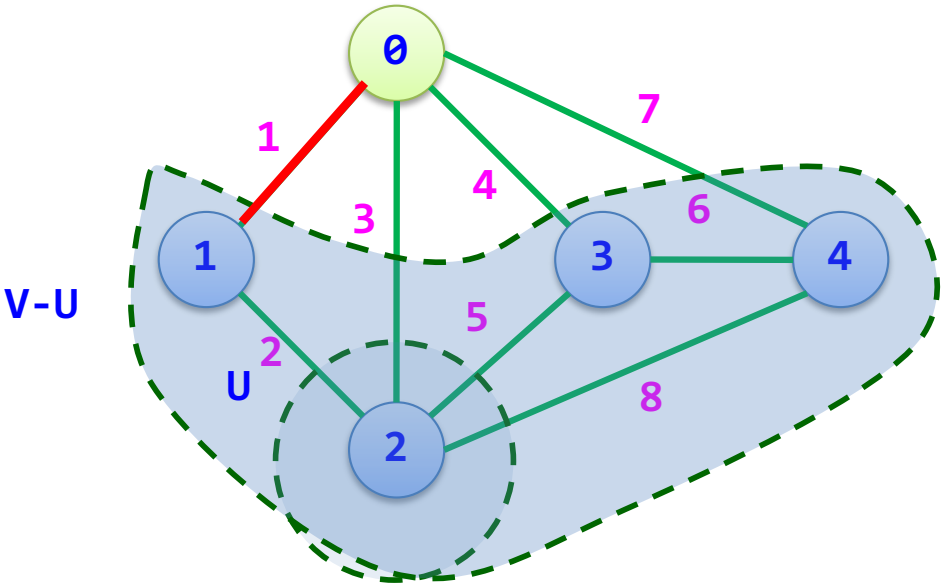


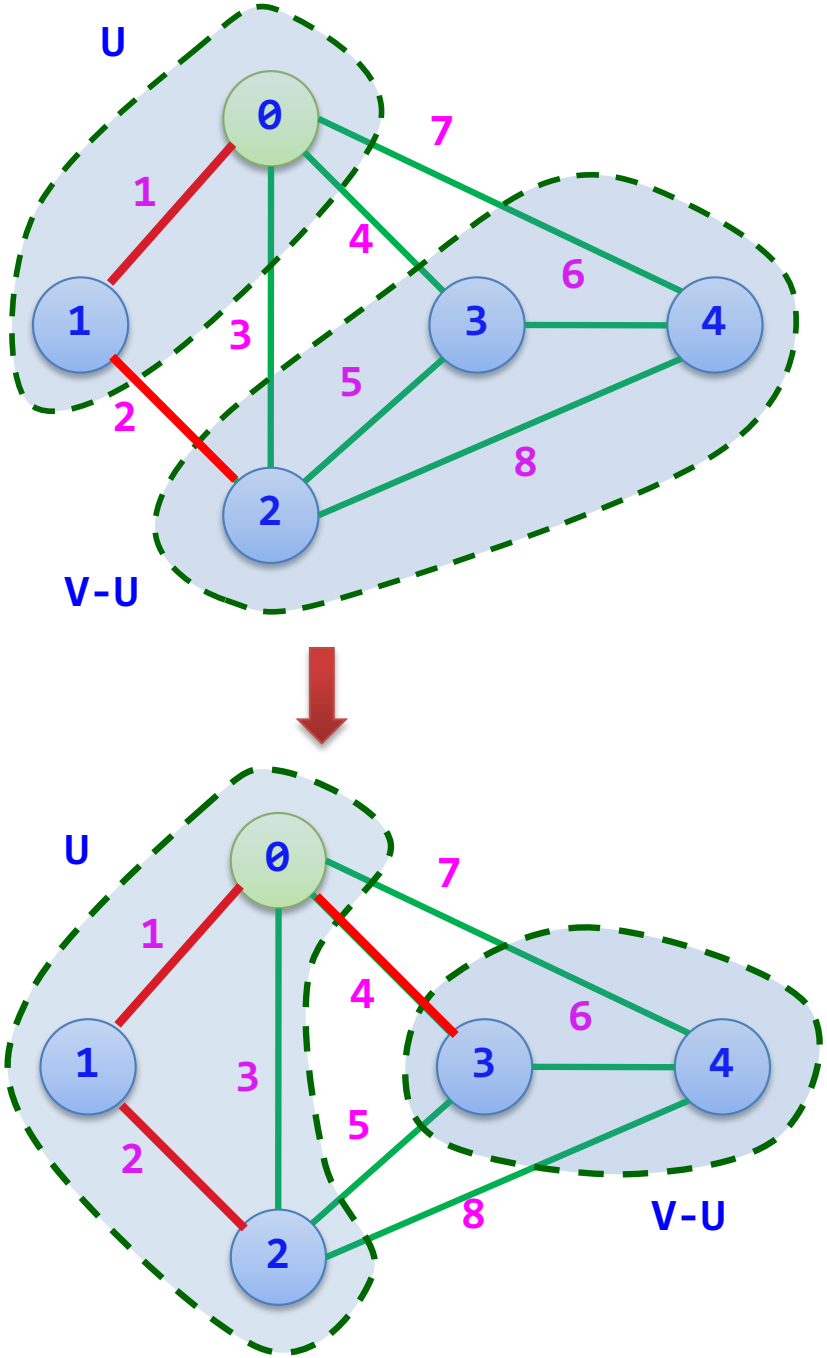
将U和V-U之间的所有边称为割集

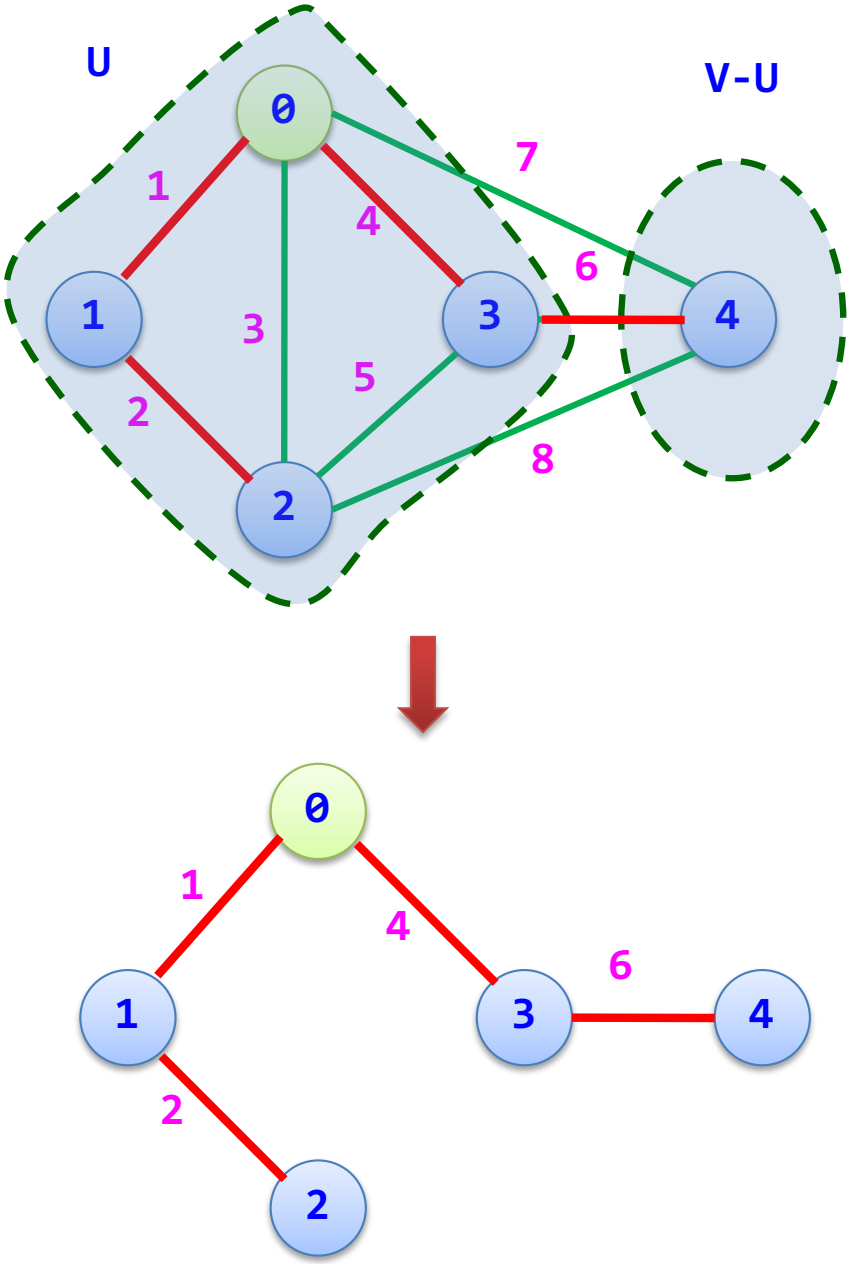
移动的顶点k是割集中的最小边（*,k）

示例






U
0
1
2
3
4
7
1
4
6
3
5
2
8
V-U
U
0
1
2
3
4
7
1
4
6
3
5
2
8
V-U

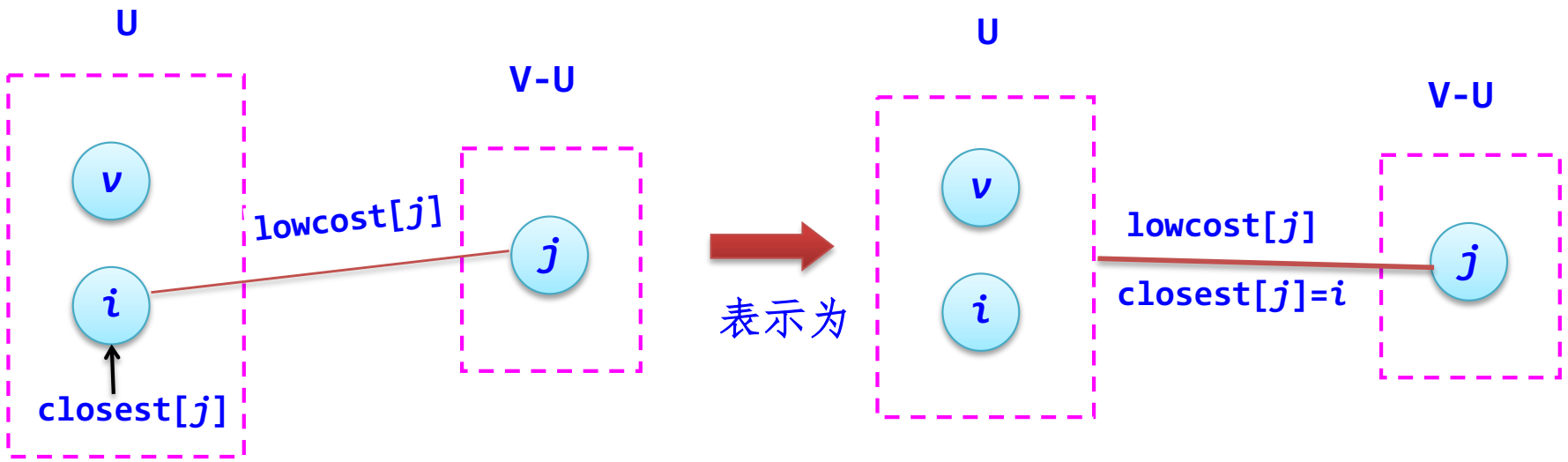


构造的最小生成树

2．普里姆算法设计

采用邻接矩阵存储图。为了记录V-U中每个顶点j（j∈V-U）到U的最小边，建立两个数组closest和lowcost：

- closest[j]表示该最小边在U中的顶点。
- lowcost[j]表示该边的权值。



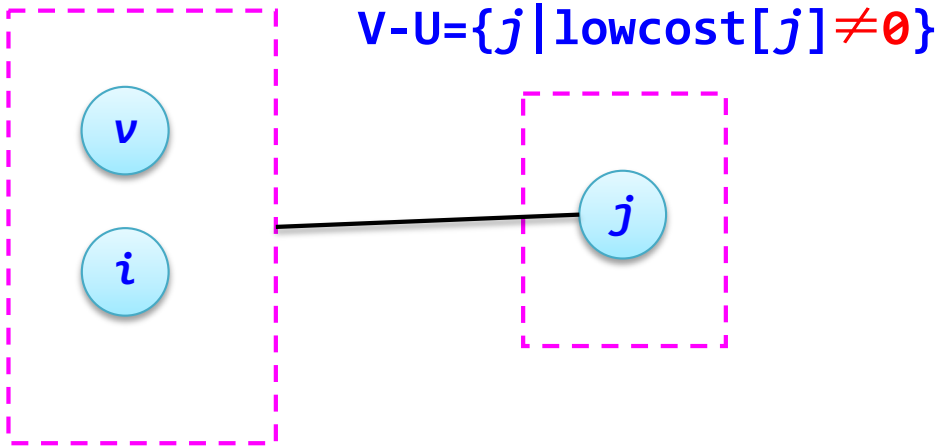
V-U中顶点j到U中的最小边为(i, j)

将V-U中顶点j到整个U的最小边表示为(j,closest[j]):lowcost[j]

对于任意顶点i，如何知道它属于集合U还是集合V-U呢？

U={i|lowcost[i]=0}

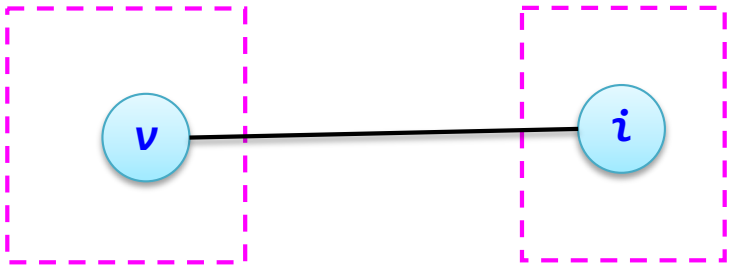
V-U={j|lowcost[j]≠0}



初始化

初始时，U中只有一个顶点v，其他顶点i均在V-U中。

U={v|lowcost[v]=0}　V-U={i|lowcost[i]≠0}

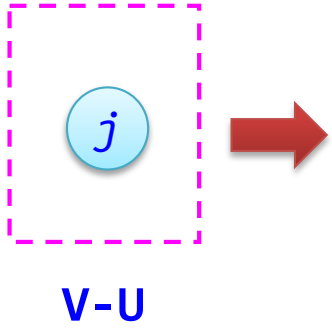


- 如果（v，i）有一条边，它就是i到U的最小边，置closest[i]=v，lowcost[i]=g.edges[v][i]。
- 如果（v，i）没有边，不妨认为有一条权为∞的边，同样置closest[i]=v，lowcost[i]=g.edges[v][i]

此时恰好有g.edges[v][i]为∞，看出邻接矩阵为什么这样表示的原因。

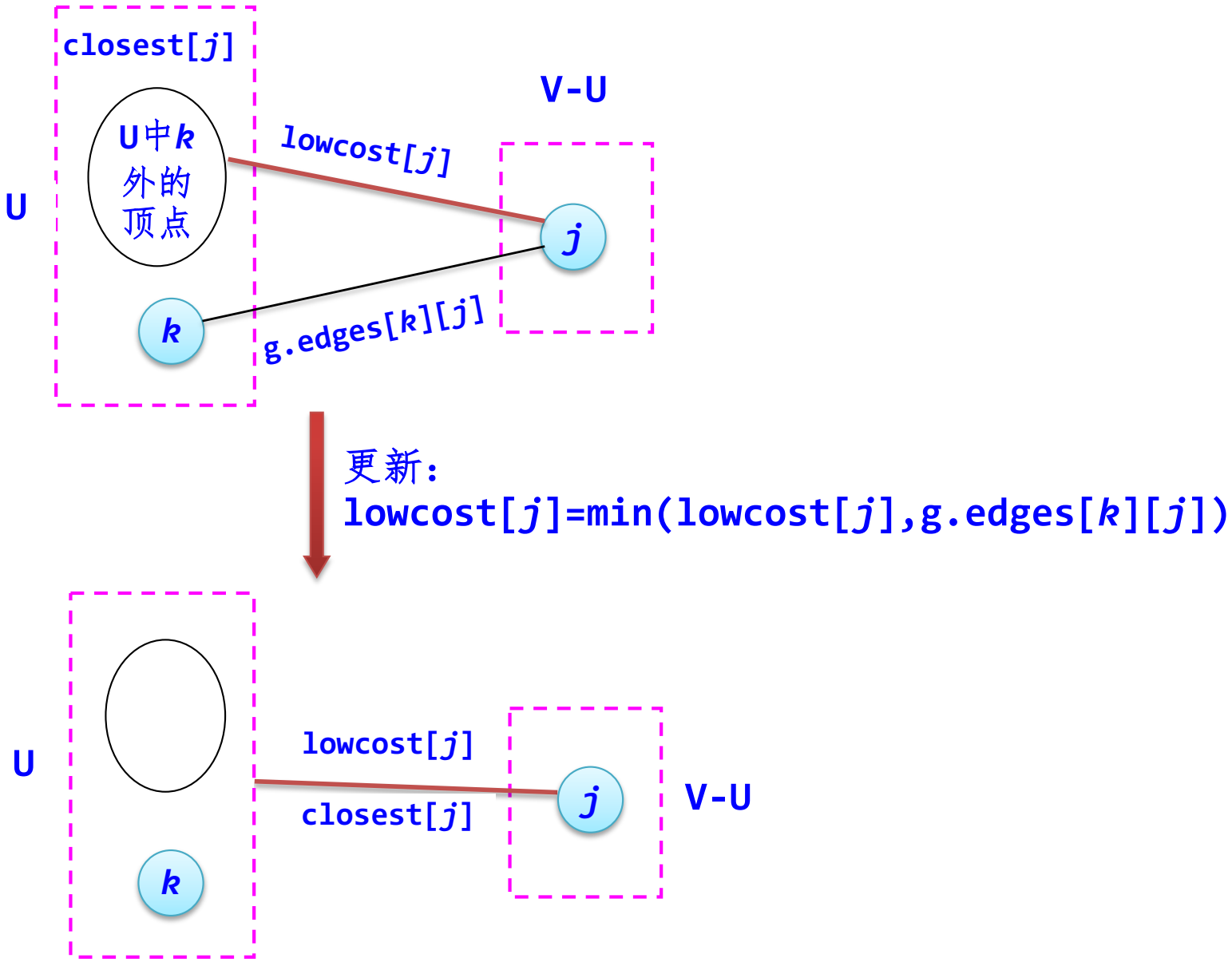
查找V-U中最小边的顶点k

V-U={j|lowcost[j]≠0}




```
int min=INF;
int k=-1;                          //k记录最近顶点的编号
for (int j=0;j<g.n;j++) //在(V-U)中找出离U最近的顶点k
    if (lowcost[j]!=0 && lowcost[j]<min)
    {  min=lowcost[j];
       k=j;
    }
```

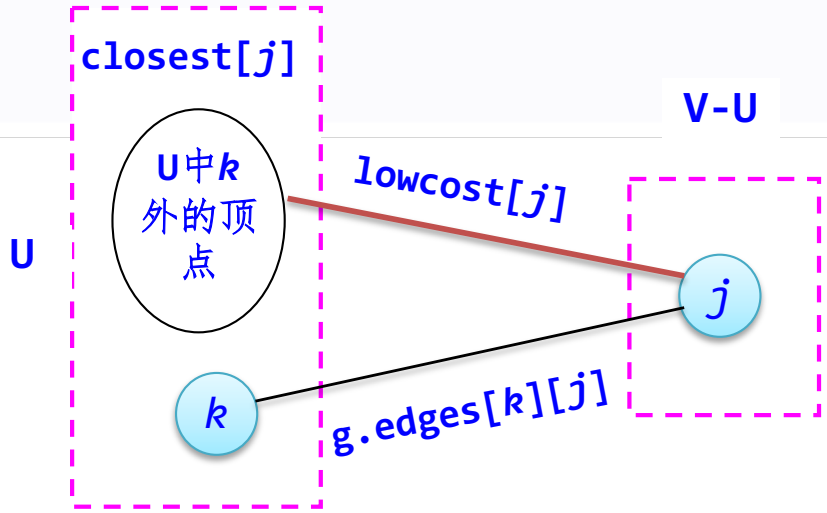
将顶点k移动到U后的调整



```
void Prim(MatGraph g,int v)              //Prim算法输出的最小生树
{  int lowcost[MAXV];                    //建立数组lowcost
   int closest[MAXV];                    //建立数组closest
   for (int i=0;i<g.n;i++)               //给lowcost[]和closest[]置初值
   {  lowcost[i]=g.edges[v][i];
      closest[i]=v;
   }
```


初始化阶段

```
    for (int i=1;i<g.n;i++)                    //找出(n-1)个顶点
    {  int min=INF;
       int k=-1;                               //k记录最近顶点的编号
       for (int j=0;j<g.n;j++)                 //在(V-U)中找出离U最近的顶点k
          if (lowcost[j]!=0 && lowcost[j]<min)
          {  min=lowcost[j];
             k=j;
          }
       cout << "  边(“<<closest[k]<<",”<<k<< "),权为”<<min<<endl;
       lowcost[k]=0;                           //标记k已经加入U
       for (int j=0;j<g.n;j++)                 //修改数组lowcost和closest
          if (lowcost[j]!=0 && g.edges[k][j]<lowcost[j])
          {  lowcost[j]=g.edges[k][j];
             closest[j]=k;
          }
    }
}
```



算法特点

- 上述普里姆算法中有两重for循环，所以时间复杂度为$O(n^2)$，其中n为图的顶点个数。
- 由于与e无关，所以普里姆算法特别适合于稠密图求最小生成树。

8.5.3. 克鲁斯卡尔算法

1. 克鲁斯卡尔算法过程

克鲁斯卡尔（Kruskal）算法是一种按权值的递增次序选择合适的边来构造最小生成树的方法。假设G=(V，E)是一个具有*n*个顶点的带权连通图，T=(U，TE)是G的最小生成树，则构造最小生成树的步骤如下：

（1）置U的初值等于V（即包含有G中的全部顶点），TE的初值为空集（即图T中每一个顶点都构成一个分量）。

（2）将图G中的边按权值从小到大的顺序依次选取：若选取的边未使生成树T形成回路，则加入TE；否则舍弃，直到TE中包含*n*-1条边为止。

2．克鲁斯卡尔算法设计

关键是如何判断选择的边是否与生成树中已有边形成回路？



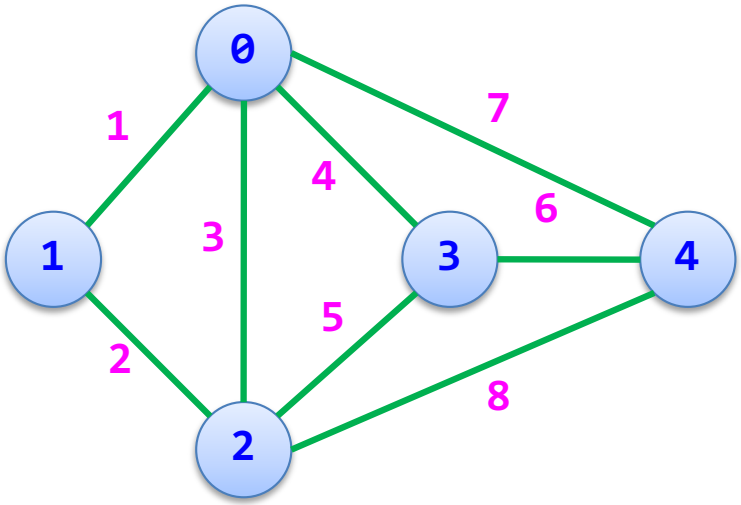
设置一个辅助数组vset[0..*n*-1]，其元素vset[*i*]代表顶点*i*所属的连通分量的编号（同一个连通分量中所有顶点的vset值相同）。

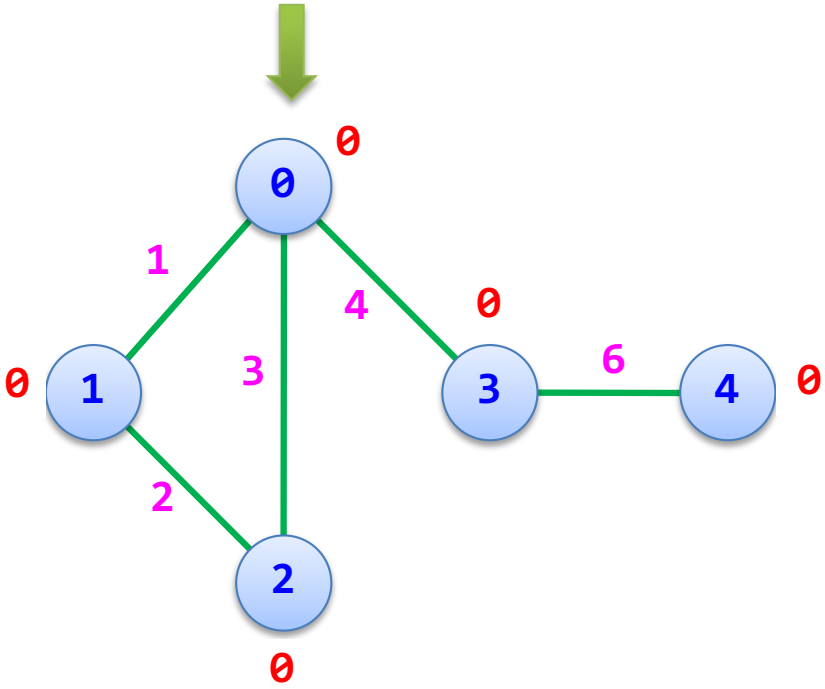
- 初始时T中只有n个顶点，没有任何边，每个顶点i看成一个连通分量，该连通分量的编号就是i，即vset[i]=i。
- 将图中所有边按权值递增排序，从前向后选边（保证总是选择权值最小的边），当选择一条边（u_1，v_1），求出这两个顶点所属连通分量的编号分别为sn_1和sn_2：

- 若$sn_1=sn_2$，说明顶点u_1和v_1属于同一个连通分量 ⇨ 不能添加该边。
- 若$sn_1 \neq sn_2$，说明顶点u_1和v_1属于不同连通分量 ⇨ 添加该边。添加后原来的两个连通分量需要合并，即将两个连通分量中所有顶点的vset值改为相同（改为sn_1或者sn_2均可）。

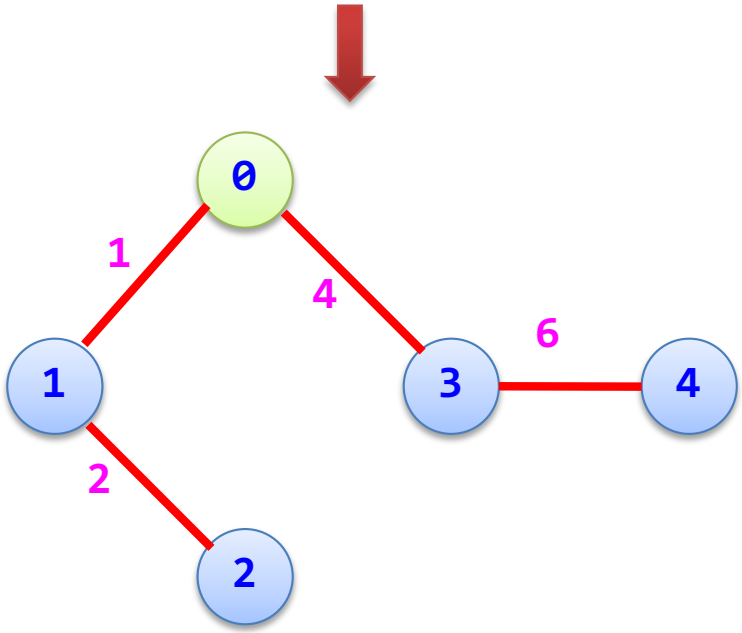
如此这样直到在T中添加n-1条边为止。

示例





构造的最小生成树



构造的最小生成树

从图G的邻接矩阵中获取所有边向量E：

- 由于是无向图，将邻接矩阵上三角部分的所有边存放在边向量E中，每一条边对应的列表为[*u*，*v*，*w*]，其中*u*、*v*分别为边的头尾顶点，*w*为边的权值。
- 对E按权值*w*递增排序后做上述操作。
- E的元素类型如下：

```
struct Edge                                    //边向量元素类型
{  int u;                                      //边的起始顶点
   int v;                                      //边的终止顶点
   int w;                                      //边的权值

   Edge(int u,int v,int w)                     //构造函数
   {  this->u=u;
      this->v=v;
      this->w=w;
   }
   bool operator<(const Edge& s) const         //重载<运算符
   {
      return w<s.w;                            //用于按w递增排序
   }
};
```

克鲁斯卡尔算法

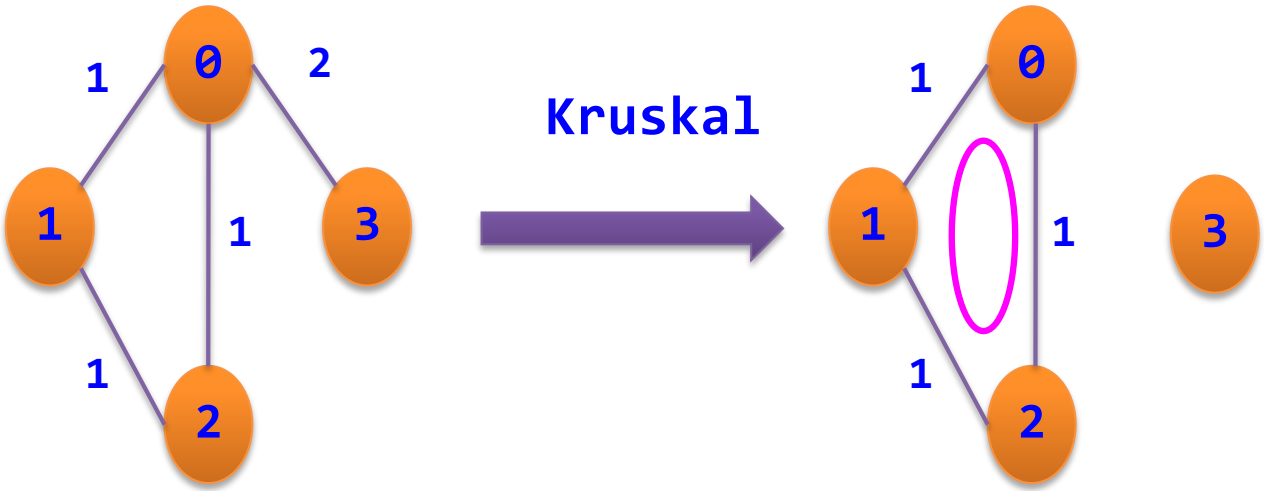
```
void Kruskal(MatGraph& g)                    //Kruskal算法输出最小生成树
{  int vset[MAXV];                           //建立数组vset
   vector<Edge> E;                           //建立存放所有边的向量E
   for (int i=0;i<g.n;i++)                   //由图的邻接矩阵g产生边向量E
      for (int j=0;j<g.n;j++)
         if (g.edges[i][j]!=0 && g.edges[i][j]!=INF && i<j)
            E.push_back(Edge(i,j,g.edges[i][j]));
   sort(E.begin(),E.end());                  //对E按权值递增排序
   for (int i=0;i<g.n;i++) vset[i]=i;        //初始化辅助数组
```

```
    int k=1;                                    //k是当前构造生成树第几条边,初值为1
    int j=0;                                    //E中边的下标,初值为0
    while (k<g.n)                               //生成的边数小于n时循环
    {  int u1=E[j].u;
       int v1=E[j].v;                           //取一条边的起始和终止顶点
       int sn1=vset[u1];
       int sn2=vset[v1];                        //分别得到两个顶点所属的集合编号
       if (sn1!=sn2)                            //两顶点属不同集合,取该边
       {  cout<<"  边(“<<u1<<",”<<v1<<"),权为" << E[j].w << endl;
          k++;                                  //生成边数增1
          for (int i=0;i<g.n;i++)               //两个集合统一编号
             if (vset[i]==sn2)                  //集合编号为sn2的改为sn1
                vset[i]=sn1;
       }
       j++;                                     //扫描下一条边
    }
}
```



一个带权连通图中所有权值最小的边一定会出现在所有的最小生成树中？

不一定！





对某个带权连通图构造最小生成树，以下说法中正确的是（）。

Ⅰ.该图的所有最小生成树的总代价一定是唯一的 √

Ⅱ.该图的最小生成树是唯一的 ×

Ⅲ.用Prim算法从不同顶点开始构造的所有最小生成树一定相同 ×

Ⅳ.使用Prim和Kruskal算法得到的最小生成树总不相同 ×

A.仅Ⅰ　　B.仅Ⅱ　　C.仅Ⅰ、Ⅲ　　仅Ⅱ、Ⅳ

A

3*．改进的克鲁斯卡尔算法设计

连通分量



并查集中的一个子集

利用并查集的克鲁斯卡尔算法

```
int parent[MAXV];                    //并查集存储结构
int rnk[MAXV];                       //存储结点的秩

void Init(int n)                     //并查集初始化
{  for (int i=0;i<n;i++)             //顶点编号0到n-1
   {  parent[i]=i;
      rnk[i]=0;
   }
}
```

并查集初始化、查找和合并算法

```
void Kruskal1(MatGraph& g)    //改进的Kruskal算法输出最小生成树
{  vector<Edge> E;                        //建立存放所有边的向量E
   for (int i=0;i<g.n;i++)                //由图的邻接矩阵g产生边向量E
      for (int j=i+1;j<g.n;j++)
         if (g.edges[i][j]!=0 && g.edges[i][j]!=INF)
            E.push_back(Edge(i,j,g.edges[i][j]));
   sort(E.begin(),E.end());               //对E按权值递增排序
   Init(g.n);                             //并查集初始化
```

```
    int k=1;                          //k表示当前构造生成树的第几条边,初值为1
    int j=0;                          //E中边的下标,初值为0
    while (k<g.n)                     //生成的边数小于n时循环
    {  int u1=E[j].u;
       int v1=E[j].v;                 //取一条边的起始和终止顶点
       int sn1=Find(u1);
       int sn2=Find(v1);              //分别得到两个顶点所属的集合编号
       if (sn1!=sn2)                  //两顶点属不同集合,取该边
       {  cout<<"  边(“<<u1<<",”<<v1<<"),权为”<<E[j].w<<endl;
          k++;                        //生成边数增1
          Union(sn1,sn2);                    //合并
       }
       j++;                           //扫描下一条边
    }
}
```

改进克鲁斯卡尔算法的时间复杂度为$O(e\log_2 e)$。由于与n无关，所以克鲁斯卡尔算法特别适合于稀疏图求最小生成树。

8.5.4. 最小生成树算法比较

比较维度	Prim 算法	Kruskal 算法
基础策略	顶点驱动 (Vertex-driven)	边驱动 (Edge-driven)
中间状态	始终是一棵连通的树	是一片不断合并的森林 (多个连通分量)
核心数据结构	优先队列	并查集
判断环的方式	自动避免 只选取一端在树外的边	依赖并查集检测 两端点是否在同一集合
最佳适用图型	稠密图	稀疏图

8.6 最短路径

8.6.1 最短路径的概念

不带权图

- 从一顶点到另一顶点存在着一条路径时，称该路径长度为该路径上所经过的边的数目，它等于该路径上的顶点数减1。
- 从一顶点到另一顶点可能存在着多条路径，每条路径上所经过的边数可能不同，即路径长度不同，把路径长度最短（即经过的边数最少）的那条路径叫做最短路径，其路径长度称为最短路径长度或最短距离。

采用广度优先遍历可以求最短路径

带权图

- 把一条路径上所经边的权值之和定义为该路径的路径长度或称带权路径长度。
- 从源点到终点可能不止一条路径，把带权路径长度最短的那条路径称为最短路径，其路径长度（权值之和）称为最短路径长度或者最短距离。

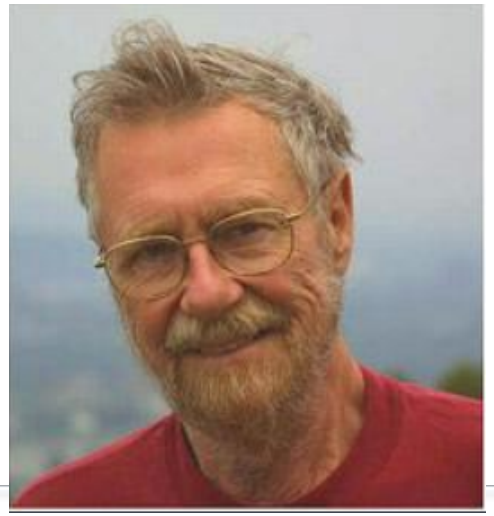


采用广度优先遍历可以求最短路径：不适合

最短路径算法应用场景

- 地图导航与路径规划（LBS 服务）：
- 计算机网络与路由协议：
  - OSPF（开放式最短路径优先）协议
- 物流与供应链优化
- 游戏开发中的 AI 寻路
  - A* 算法（A-Star），控制 NPC 在复杂地形中寻找通往玩家的最短且无障碍的路径。
- 社交网络分析：

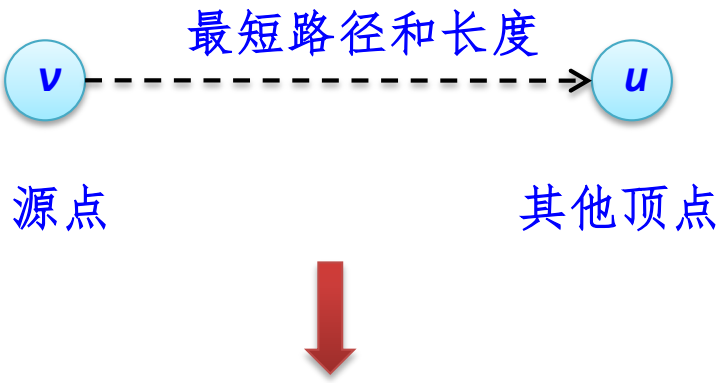
8.6.2 狄克斯特拉算法



Edsger Wybe Dijkstra
1930年5月11日~2002年8月6日

- 提出“goto有害论”
- 提出信号量和PV原语
- 解决了“哲学家聚餐”问题
- Dijkstra最短路径算法和银行家算法的创造者
- 第一个Algol 60编译器的设计者和实现者
- THE操作系统的设计者和开发者
- 1972年获得图灵奖
- 与D.E.Knuth并称为我们这个时代最伟大的计算机科学家的人，与癌症抗争多年，于2002年8月6日在荷兰Nuenen自己的家中去世，享年72岁

1．狄克斯特拉算法过程

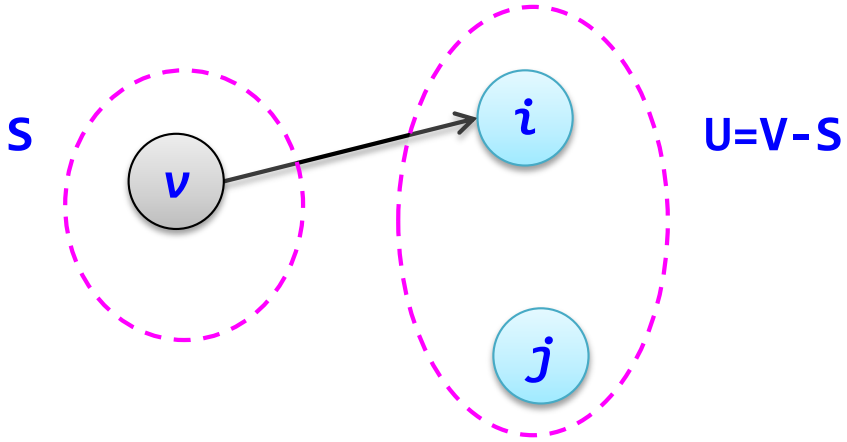


单源最短路径算法

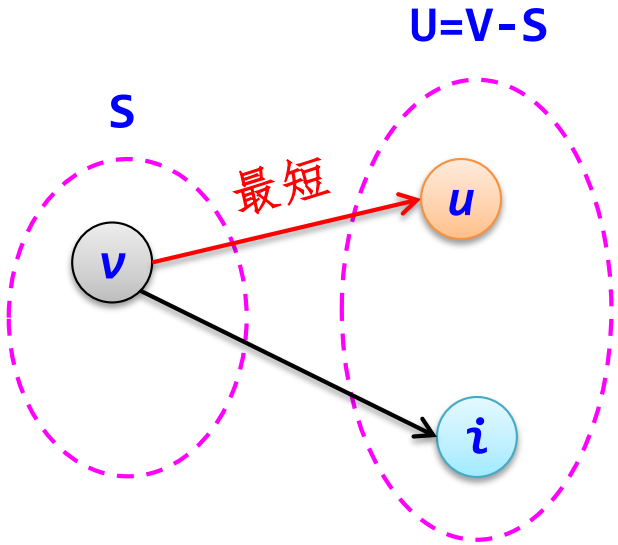
给定一个带权图G和一个起始点即源点v，狄克斯特拉算法的具体步骤如下：

（1）初始化：顶点集S只包含源点，即S={v}，顶点集U包含除v外的其他顶点。

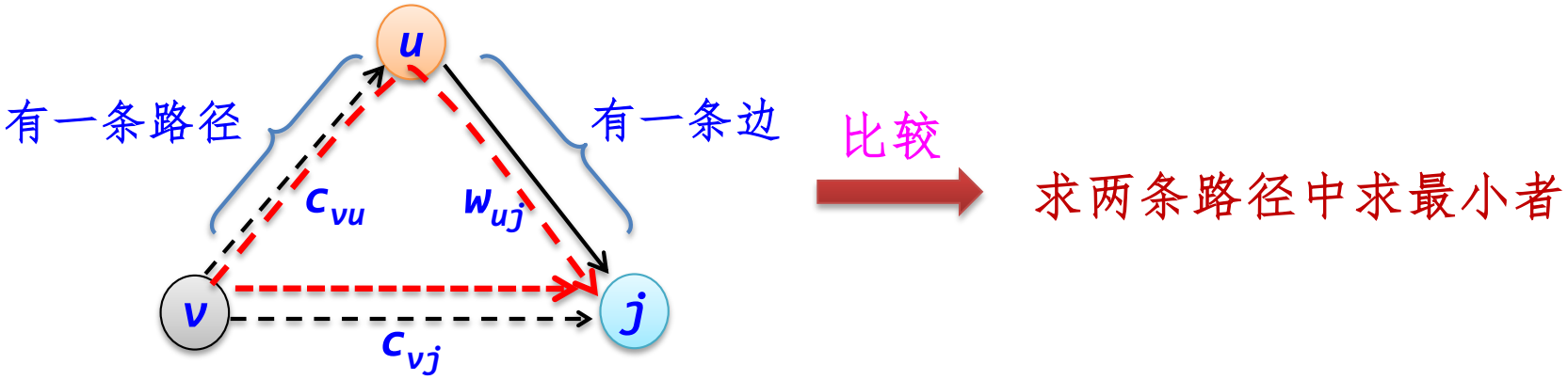
- 顶点v到自己的最短路径长度为0。
- 源点v到U中顶点i的最短路径长度为该边权值（若源点v到顶点i有边<v，i>）。
- 源点v到U中顶点i的最短路径长度为∞（若源点v到顶点i没有边，此时认为有一条长度为∞的最短路径）。



（2）从U中选取一个顶点*u*，它是源点*v*到U中最短路径长度最小的顶点，然后把顶点*u*加入S中（此时求出了源点*v*到顶点*u*的最短路径长度）。



（3）以顶点u为新考虑的中间点，修改顶点u的出边邻接点j的最短路径长度，此时源点v到顶点j的最短路径有两条，即一条经过顶点u，一条不经过顶点u：

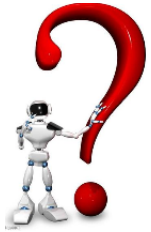


（4）重复步骤（2）和（3）直到S包含所有的顶点即U为空。

2．狄克斯特拉算法设计

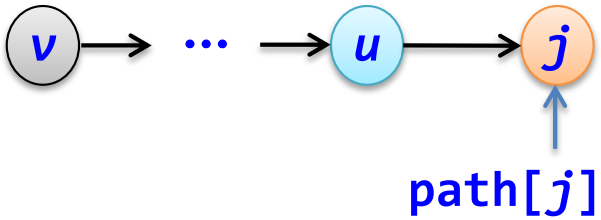
狄克斯特拉算法设计要点：

- 判断顶点**i**属于哪个集合，设置一个数组**S**，**S[i]=1**表示顶点**i**属于**S**集合，**S[i]=0**表示顶点**i**属于**U**集合。
- 保存最短路径长度，由于源点**v**是已知的，只需要设置一个数组**dist[0..*n*-1]**，**dist[i]**用来保存从源点**v**到顶点**i**的最短路径长度。
- 保存最短路径，设置一个数组**path[0..*n*-1]**，其中**path[i]**存放从源点**v**到顶点**i**的最短路径。



为什么能够用一个一维数组保存多条最短路径呢？

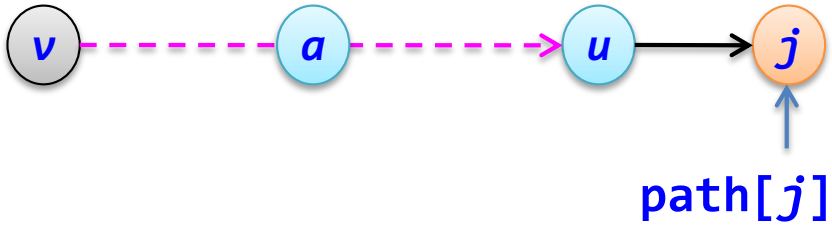
path[j]只保存源点v到顶点j的最短路径上顶点j的前驱顶点



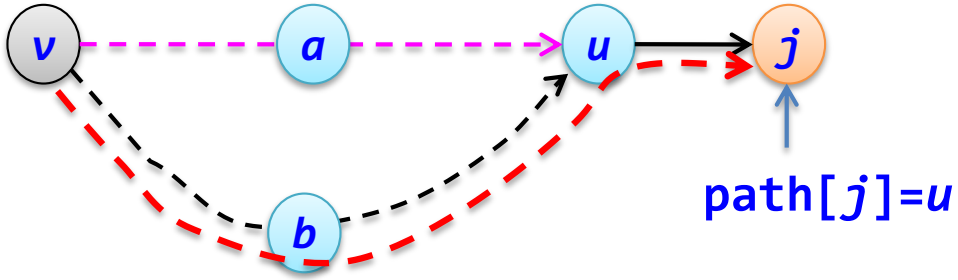
能够这样表示路径的前提

若该路径是v ⇨ j的最短路径，则其中v → u部分一定是v ⇨ u的最短路径。

证明



假设 $v \to a \to u \to j$ 是v到j的最短路径
但 $v \to a \to u$ 不是v到u的最短路径



该路径更短，与假设矛盾！

path表示

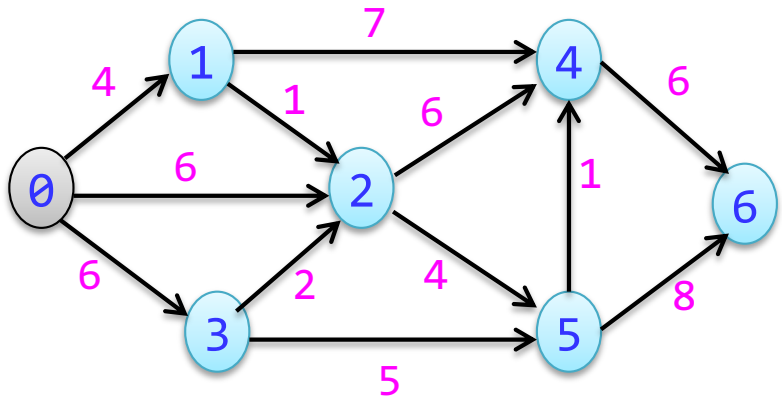


由path反推最短路径

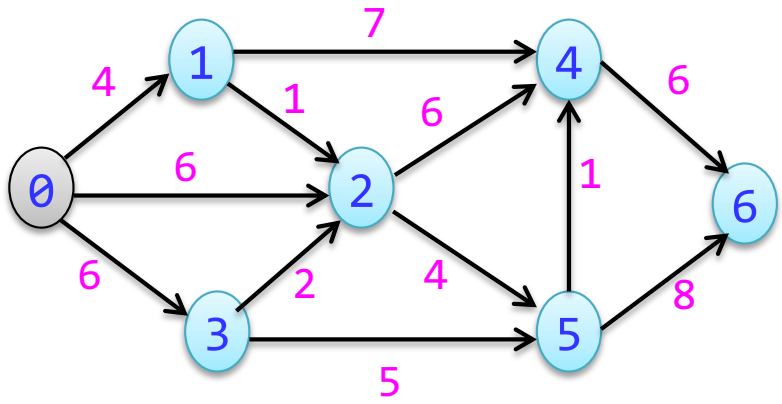
path[j]=u
path[u]=w
path[w]=v（到源点为止）

逆路径：j u w v
反向：v → w → u → j

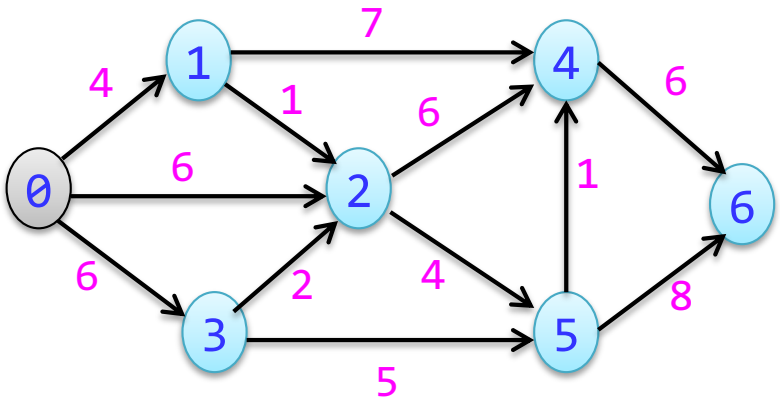
示例



S	U	dist[]	path[]
		0 1 2 3 4 5 6	0 1 2 3 4 5 6
{0}	{1,2,3,4,5,6}	{0, 4, 6, 6, ∞, ∞, ∞}	{0, 0, 0, 0, -1, -1, -1}
		最小的顶点：1	
{0,1}	{2,3,4,5,6}	{0, 4, 5, 6, 11, ∞, ∞}	{0, 0, 1, 0, 1, -1, -1}
		最小的顶点：2	
{0,1,2}	{3,4,5,6}	{0, 4, 5, 6, 11, 9, ∞}	{0, 0, 1, 0, 1, 2, -1}



S	U	dist[]	path[]
		0 1 2 3 4 5 6	0 1 2 3 4 5 6
{0,1,2}	{3,4,5,6}	{0, 4, 5, 6, 11, 9, ∞}	{0, 0, 1, 0, 1, 2, -1}
		最小的顶点：3	
{0,1,2,3}	{4,5,6}	{0, 4, 5, 6, 11, 9, ∞}	{0, 0, 1, 0, 1, 2, -1}
		最小的顶点：5	
{0,1,2,3,5}	{4,6}	{0, 4, 5, 6, 10, 9, 17}	{0, 0, 1, 0, 5, 2, 5}



S	U	dist[]	path[]
		0 1 2 3 4 5 6	0 1 2 3 4 5 6
{0,1,2,3,5}	{4,6}	{0, 4, 5, 6, 10, 9, 17}	{0, 0, 1, 0, 5, 2, 5}
		最小的顶点：4	
{0,1,2,3,5,4}	{6}	{0, 4, 5, 6, 10, 9, 16}	{0, 0, 1, 0, 5, 2, 4}
		最小的顶点：6	
{0,1,2,3,5,4,6}	{}	{0, 4, 5, 6, 10, 9, 16}	{0, 0, 1, 0, 5, 2, 4}

最终结果

利用dist和path求最短路径长度和最短路径

❶ 求0 ⇨ 6的最短路径长度：

```
      0  1  2  3  4   5  6
dist={0, 4, 5, 6, 10, 9, 16}
```

从顶点0 ⇨6的最短路径长度为16

❷ 求0 ⇨ 6的最短路径：

```
      0  1  2  3   4   5   6
path={0, 0, 1, 0,  5,  2,  4}
```

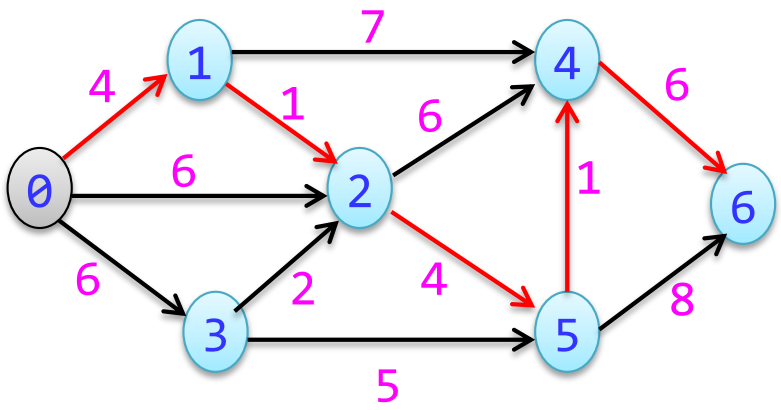
path[6]=4

path[4]=5

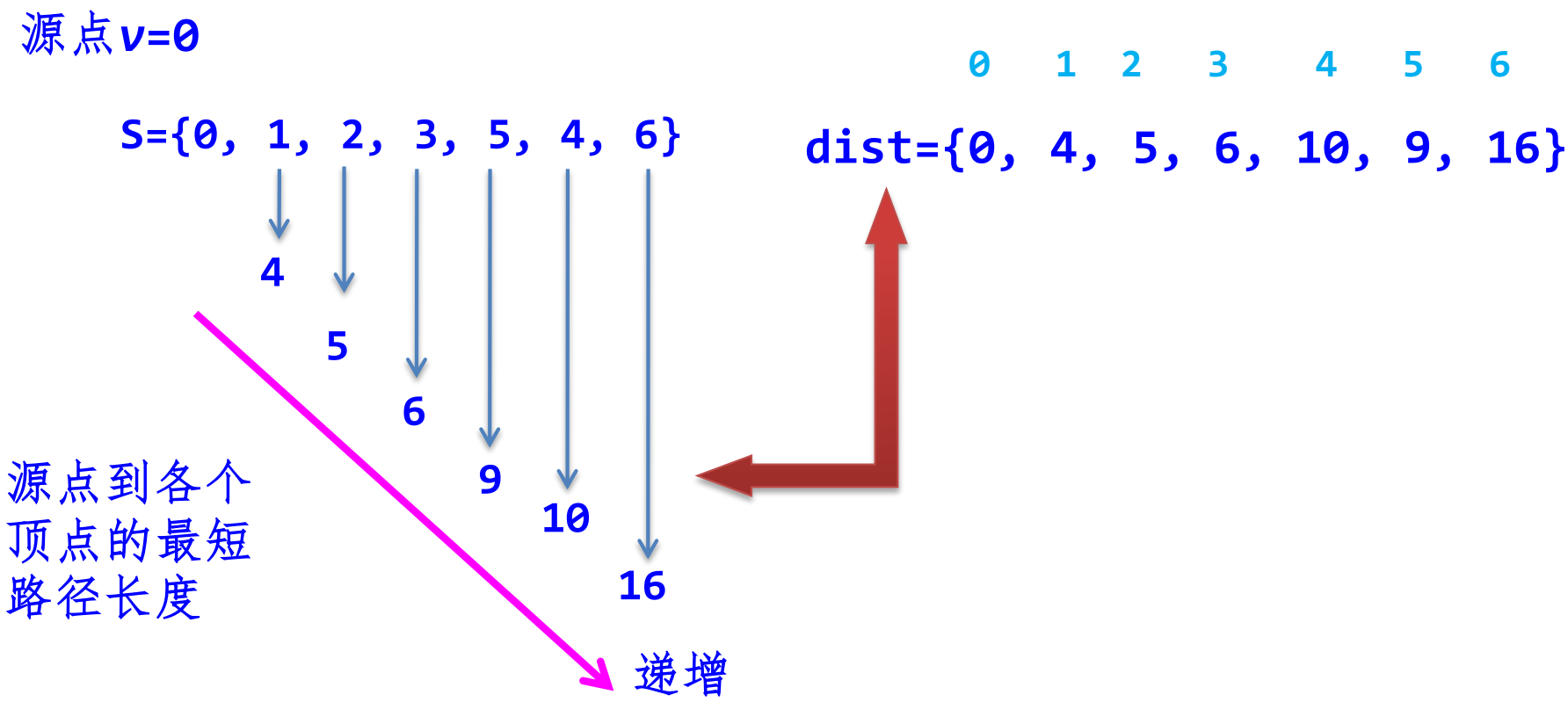
path[5]=2

path[2]=1

path[1]=0到源点



观察求解结果

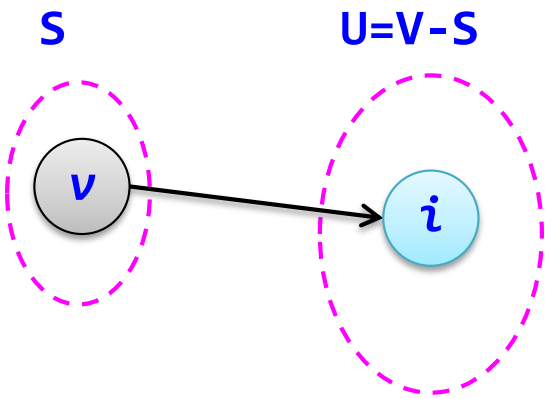


结论：

❶ 按顶点进入S的先后顺序，最短路径长度越来越长。

❷ 一个顶点一旦进入S后，其最短路径长度不再改变（调整）。

```
void Dijkstra(MatGraph& g,int v)      //求从v到其他顶点的最短路径
{  int dist[MAXV];                    //建立dist数组
   int path[MAXV];                    //建立path数组
   int S[MAXV];                               //建立S数组
   for (int i=0;i<g.n;i++)
   {  dist[i]=g.edges[v][i];          //距离初始化
      S[i]=0;                         //S[]置空
      if (g.edges[v][i]!=0 && g.edges[v][i]<INF)
         path[i]=v;                   //v到i有边时，置i的前驱顶点为v
      else
         path[i]=-1;                  //v到i没边时，置i的前驱顶点为-1
   }
   S[v]=1;                            //源点编号v放入S中
```



- dist[i]=g.edges[v][i]
- path[i]=v

```
    int mindis,u=-1;
    for (int i=0;i<g.n-1;i++)          //循环向S中添加n-1个顶点
    {  mindis=INF;                     //mindis置最小距离初值
       for (int j=0;j<g.n;j++)         //选取不在S中且具有最小距离的顶点u
          if (S[j]==0 && dist[j]<mindis)
          {  u=j;
             mindis=dist[j];
          }
       S[u]=1;                         //顶点u加入S中
       for (int j=0;j<g.n;j++)         //修改不在s中的顶点的距离
          if (S[j]==0)
             if (g.edges[u][j]<INF && dist[u]+g.edges[u][j]<dist[j])
            {  dist[j]=dist[u]+g.edges[u][j];
               path[j]=u;
            }
    }
    DispAllPath(dist,path,S,v,g.n);   //输出所有最短路径及长度
}
```

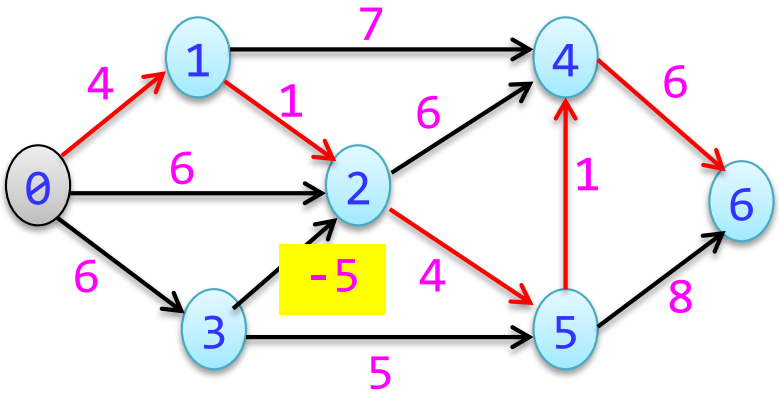
已经在S中的顶点不再修改其最短路径长度

算法的时间复杂度为$O(n^2)$。

```
void DispAllPath(int dist[],int path[],int S[],int v,int n)
//输出从顶点v出发的所有最短路径
{  for (int i=0;i<n;i++)                    //循环输出从顶点v到i的路径
     if (S[i]==1 && i!=v)
     {  vector<int> apath;                  //存放一条最短逆路径
        printf("  从%d到%d最短路径长度为: %d\t路径: ",v,i,dist[i]);
        apath.push_back(i);                 //添加终点i
        int pre=path[i];
        while (pre!=v)
        {  apath.push_back(pre);
           pre=path[pre];
        }
        printf("%d",v);                     //先输出起点v
        for (int k=apath.size()-1;k>=0;k--)
           printf("->%d",apath[k]);  //再反向输出路径中其他顶点
        printf("\n");
     }
     else printf("  从%d到%d没有路径\n",v,i);
}
```

提示

Dijkstra算法是否适合含负权的图求最短路径



0 1 2 3 4 5 6

dist={0, 4, 5, 6, 10, 9, 16}

path={0, 0, 1, 0, 5, 2, 4}

S	U	dist[]
		0 1 2 3 4 5 6
{0}	{1,2,3,4,5,6}	{0, 4, 6, 6, ∞, ∞, ∞}
{0,1}	{2,3,4,5,6}	{0, 4, 5, 6, 11, ∞, ∞}
{0,1,2}	{3,4,5,6}	{0, 4, 5, 6, 11, 9, ∞}
{0,1,2,3}	{4,5,6}	{0, 4, 1 6, 11, 9, ∞}
{0,1,2,3,5}	{4,6}	{0, 4, 5, 6, 10, 9, 17}
{0,1,2,3,5,4}	{6}	{0, 4, 5, 6, 10, 9, 16}
{0,1,2,3,5,4,6}	{}	{0, 4, 5, 6, 10, 9, 16}



Dijkstra算法是（ ）方法求出图中从源点到其余顶点最短路径的。

A.按长度递减的顺序求出图的某顶点到其余顶点的最短路径

B.按长度递增的顺序求出图的某顶点到其余顶点的最短路径 √

C.通过深度优先遍历求出图中某顶点到其余顶点的最短路径

D.通过广度优先遍历求出图中某顶点到其余顶点的最短路径

加入S集合的顶点：



- 最短路径不再改变
- 越后加入的顶点，dist越长

3*．改进的狄克斯特拉算法设计

前面的狄克斯特拉算法从两个方面进行优化，这里仅仅输出最短路径长度：

- 采用邻接表存储图，可以更快地查找到顶点u的所有邻接点并进行调整，时间为O(MAX(图中顶点的出度))。
- 求目前一个最短路径长度的顶点u时，Dijkstra1算法采用简单比较方法，可以改为采用优先队列（小根堆）求解。由于最多e条边进队，对应的时间为$O(\log_2 e)$。



算法的最坏时间复杂度为$O(e\log_2 e)$

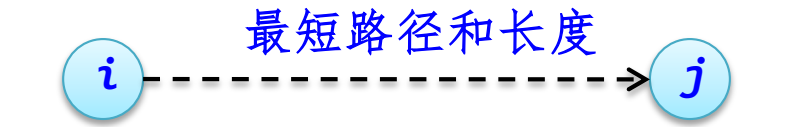
8.6.3 弗洛伊德算法

Robert W.Floyd（1936—2001）



- 弗洛伊德1936年6月8日生于纽约。说他“自学成才”并不是说他没有接受过高等教育，他是芝加哥大学的毕业生，但学的不是数学或电气工程等与计算机密切相关的专业，而是文学，1953年获得文学士学位。
- 弗洛伊德通过勤奋学习和深入研究，在计算机科学的诸多领域：算法，程序设计语言的逻辑和语义，自动程序综合，自动程序验证，编译器的理论和实现等方面都作出创造性的贡献
- 弗洛伊德是1978年获得图灵奖

1．弗洛伊德算法过程



任意顶点i　　任意顶点j

多源最短路径算法

- 假设有向图G=(V,E)采用邻接矩阵g表示，另外设置一个二维数组A用于存放当前顶点之间的最短路径长度，即分量$A[i][j]$表示当前顶点i到顶点j的最短路径长度。
- 弗洛伊德算法的基本思想是递推产生一个矩阵序列A_0、A_1、…、A_k、…、A_{n-1}，其中$A_k[i][j]$表示从顶点i到顶点j的路径上所经过的顶点编号不大于k的最短路径长度。

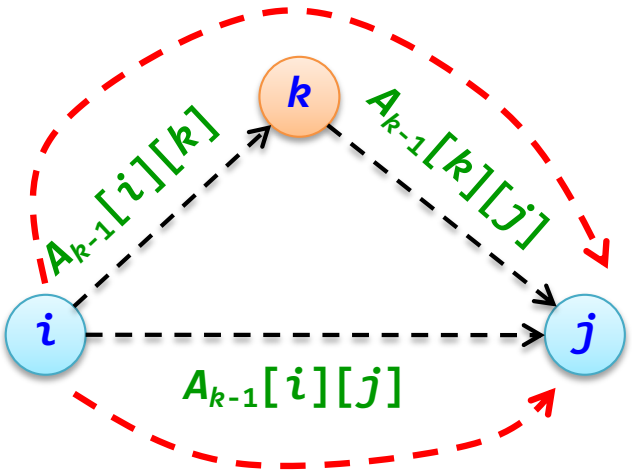


按顶点编号顺序进行迭代

归纳起来，弗洛伊德思想可用如下的表达式来描述：

$A_{-1}[i][j]=g.edges[i][j]$

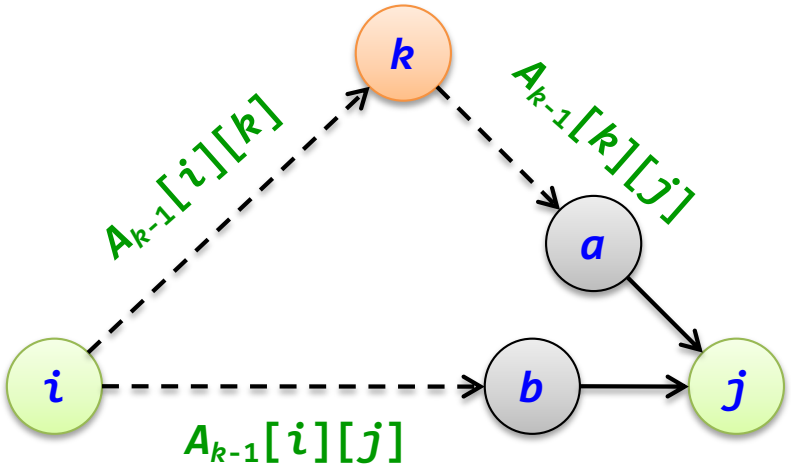
$A_k[i][j]=MIN\{A_{k-1}[i][j],A_{k-1}[i][k]+A_{k-1}[k][j]\}$　$0 \leqslant k \leqslant n-1$



两条路径中选最小者：

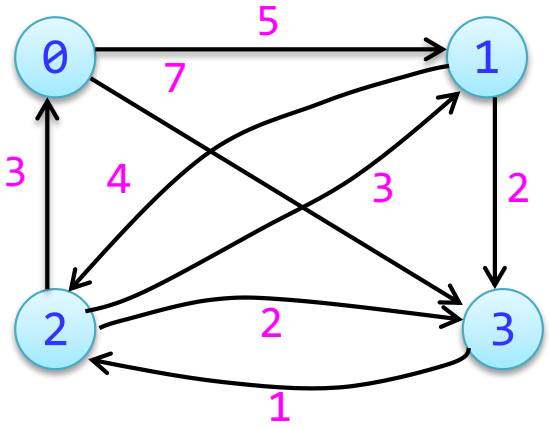
$A_k[i][j]=$ MIN {
$A_{k-1}[i][j]$,
$A_{k-1}[i][k]+A_{k-1}[k][j]$ }

- 另外用二维数组path保存最短路径，它与当前迭代的次数有关。
- $path_k[i][j]$在考虑顶点k时得到的从顶点i到顶点j的最短路径的前驱顶点。



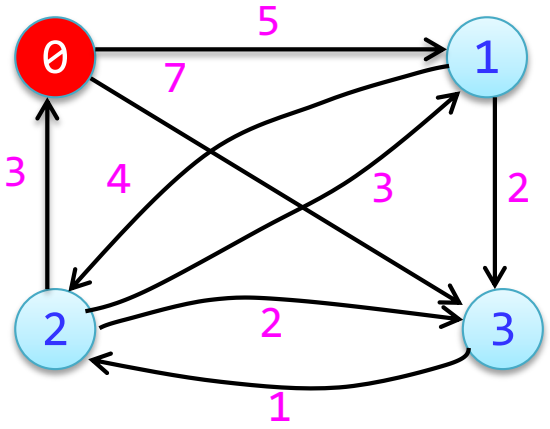
- $path_{k-1}[i][j]=b$，$path_{k-1}[k][j]=a$
- 若$A_{k-1}[i][j]>A_{k-1}[i][k]+A_{k-1}[k][j]$，选择经过顶点$k$的路径，即$path_k[i][j]=a=path_{k-1}[k][j]$。
- 否则不改变。

示例



$$\begin{bmatrix} 0 & 5 & \infty & 7 \\ \infty & 0 & 4 & 2 \\ 3 & 3 & 0 & 2 \\ \infty & \infty & 1 & 0 \end{bmatrix}$$

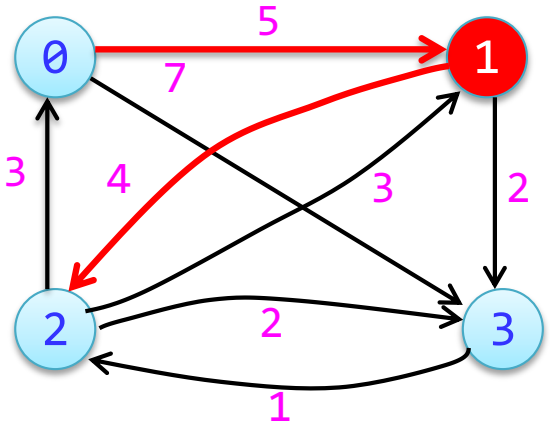
A_{-1}				$path_{-1}$			
0	5	∞	7	-1	0	-1	0
∞	0	4	2	-1	-1	1	1
3	3	0	2	2	2	-1	2
∞	∞	1	0	-1	-1	3	-1



在考虑顶点0时：

没有任何最短路径得到修改！

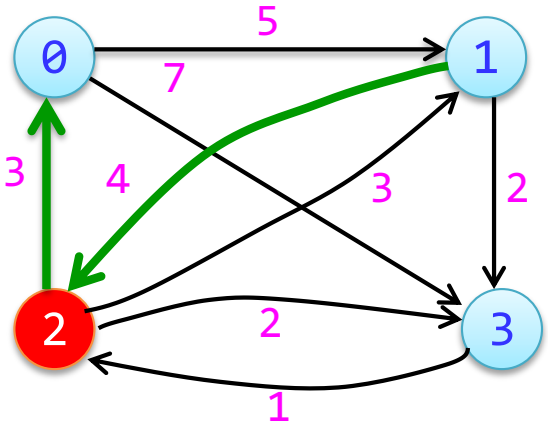
A_0				$path_0$			
0	5	∞	7	-1	0	-1	0
∞	0	4	2	-1	-1	1	1
3	3	0	2	2	2	-1	2
∞	∞	1	0	-1	-1	3	-1



在考虑顶点1时：

- 0→2：由无路径改为0→1→2，长度为9，path[0][2]改为1

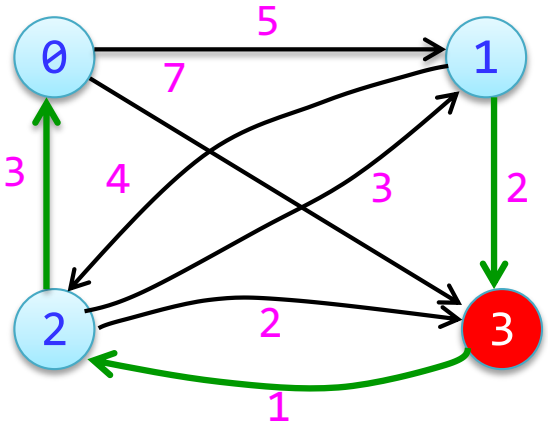
A_1				$path_1$			
0	5	9	7	-1	0	1	0
∞	0	4	2	-1	-1	1	1
3	3	0	2	2	2	-1	2
∞	∞	1	0	-1	-1	3	-1



在考虑顶点2时：

- 1→0：由无路径改为1→2→0，长度为7，path[1][0]改为2
- 3→0：由无路径改为3→2→0，长度为4，path[3][0]改为2
- 3→1：由无路径改为3→2→1，长度为4，path[3][1]改为2

A_2				$path_2$			
0	5	9	7	-1	0	1	0
7	0	4	2	2	-1	1	1
3	3	0	2	2	2	-1	2
4	4	1	0	2	2	3	-1



在考虑顶点3时：

- 0→2：由0→1→2改为0→3→2，长度为8，path[0][2]改为3
- 1→0：由1→2→0改为1→3→2→0，长度为6，path[1][0]改为2
- 1→2：由1→2改为1→3→2，长度为3，path[1][2]改为3

A_3				$path_3$			
0	5	8	7	-1	0	3	0
6	0	3	2	2	-1	3	1
3	3	0	2	2	2	-1	2
4	4	1	0	2	2	3	-1

A_3				$path_3$			
0	5	8	7	-1	0	3	0
6	0	3	2	2	-1	3	1
3	3	0	2	2	2	-1	2
4	4	1	0	2	2	3	-1

❶ 求1 ⇨ 0的最短路径长度：

由A_3数组可以直接得到两个顶点之间的最短路径长度。

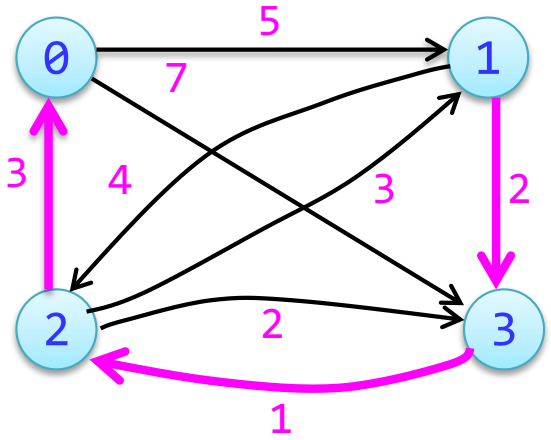
如A_3[1][0]=6，说明顶点1到0的最短路径长度为6。

A_3				$path_3$			
0	5	8	7	-1	0	3	0
6	0	3	2	2	-1	3	1
3	3	0	2	2	2	-1	2
4	4	1	0	2	2	3	-1

❷ 求1 ⇨ 0的最短路径：

```
path[1][0]=2
path[1][2]=3
path[1][3]=1（找到起点）
```

得到的顶点序列为0、2、3、1
顶点1到0的最短路径为1→3→2→0

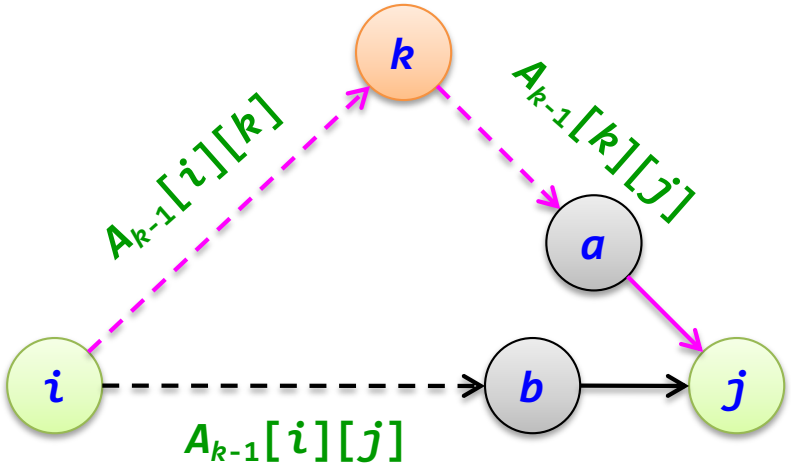


2．弗洛伊德算法设计

```
void Floyd(MatGraph& g)                    //Floyd求多源最短路径
{  int A[MAXV][MAXV];                      //建立A数组
   int path[MAXV][MAXV];                   //建立path数组

   for (int i=0;i<g.n;i++)                 //给数组A和path置初值]
      for (int j=0;j<g.n;j++)
      {  A[i][j]=g.edges[i][j];
         if (i!=j && g.edges[i][j]<INF)
            path[i][j]=i;                  //i和j顶点之间有一条边时
         else
            path[i][j]=-1;                 //i和j顶点之间没有一条边时
      }
```

```
    for (int k=0;k<g.n;k++)                          //求Ak[i][j]
    {  for (int i=0;i<g.n;i++)
           for (int j=0;j<g.n;j++)
               if (A[i][j]>A[i][k]+A[k][j])
               {  A[i][j]=A[i][k]+A[k][j];
                  path[i][j]= path[k][j];            //修改最短路径
               }
    }
    Dispath(A,path,g.n);                             //输出最短路径和长度
}
```

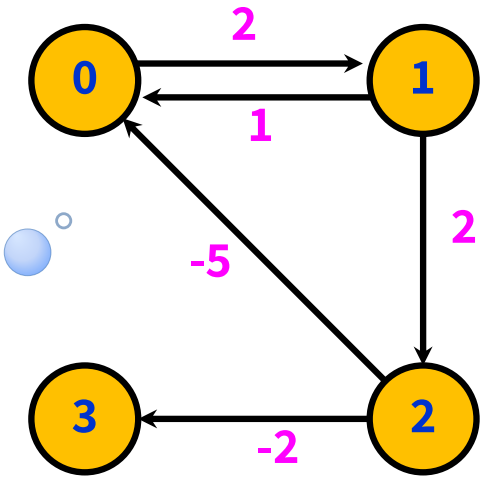


```
void Dispath(int A[][MAXV],int path[][MAXV],int n)
//输出所有的最短路径和长度
{  for (int i=0;i<n;i++)
      for (int j=0;j<n;j++)
      {  if (A[i][j]!=INF && i!=j)   //若顶点i和j之间存在路径
         {  vector<int> apath;       //存放一条i到j的最短逆路径
            printf("顶点%d到%d的最短路径长度:%d\t路径:",i,j,A[i][j]);
            apath.push_back(j);      //路径上添加终点j
            int pre=path[i][j];
            while (pre!=i)           //路径上添加中间点
            {  apath.push_back(pre); //顶点pre加入到路径中
               pre=path[i][pre];
            }
            cout << i;                          //输出起点i
            for (int k=apath.size()-1;k>=0;k--)
                printf("->%d",apath[k]);  //反向输出路径上的其他顶点
            printf("\n");
         }
      }
}
```

上述弗洛伊德算法中有三重循环，其时间复杂度为$O(n^3)$。

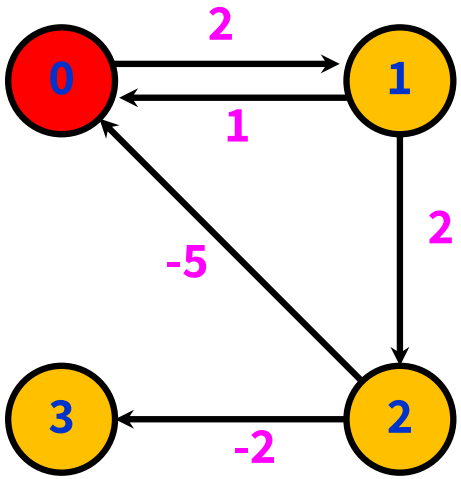


- 弗洛伊德算法适合含负权的图求最短路径
- 不适合含负权回路的图求最短路径



0	2	∞	∞
1	0	2	∞
-5	∞	0	-2
∞	∞	∞	0

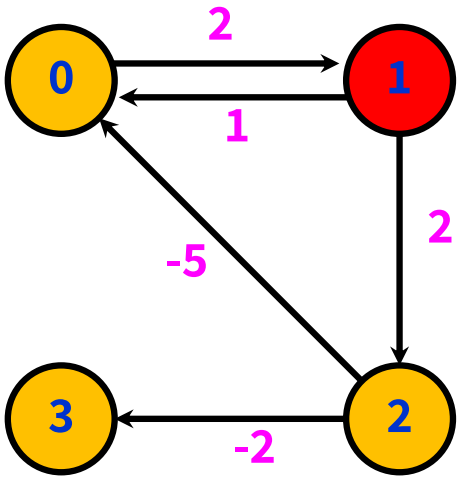
A_{-1}				$path_{-1}$			
0	2	∞	∞	-1	0	-1	-1
1	0	2	∞	1	-1	1	-1
-5	∞	0	-2	2	-1	-1	2
∞	∞	∞	0	-1	-1	-1	-1



在考虑顶点0时：

- 2→1：由无路径改为2→0→1，长度为-3，path[0][2]改为0

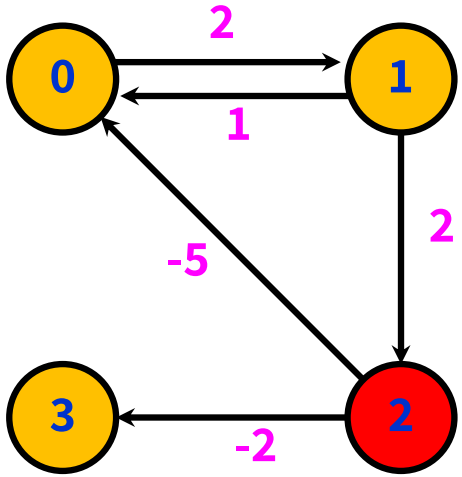
A_0				$path_0$			
0	2	∞	∞	-1	0	-1	-1
1	0	2	∞	1	-1	1	-1
-5	-3	0	-2	2	0	-1	2
∞	∞	∞	0	-1	-1	-1	-1



在考虑顶点1时：

- 0→2：由无路径改为0→1→2，长度为4，path[0][2]改为1

A_1				$path_1$			
0	2	4	∞	-1	0	1	-1
1	0	2	∞	1	-1	1	-1
-5	-3	0	-2	2	0	-1	2
∞	∞	∞	0	-1	-1	-1	-1



在考虑顶点2时：

- 0→2：由无路径改为0→1→2，长度为4，path[0][2]改为1

A_2				$Path_2$			
0	2	4	∞	-1	0	1	-1
1	0	2	∞	1	-1	1	-1
-5	-3	0	-2	2	0	-1	2
∞	∞	∞	0	-1	-1	-1	-1



以下叙述正确的是（　）。

A. 最短路径一定是简单路径 √

B. Dijkstra算法不适合有回路的带权图求最短路径

C. Dijkstra算法不适合求任意两个顶点的最短路径

D. Floyd算法求两个顶点的最短路径时，$path_{k-1}$一定是$path_k$的子集

8.6.4. 最短路径算法比较

比较维度	Dijkstra 算法	Floyd 算法
解决的问题	单源最短路径：求一个特定起点到其他所有节点的最短距离。	多源最短路径：求图中任意两个节点之间的最短距离。
算法思想	贪心算法：每次从未访问节点中选择距离起点最近的节点进行“松弛”操作，逐步向外扩展。	动态规划：通过逐步引入中间节点 k，不断更新节点i到节点j的距离。
时间复杂度	基础：$O(V^2)$ 堆优化后：$O(E*\log V)$	$O(V^3)$
空间复杂度	$O(V)$ （需要一个一维距离数组）	$O(V^2)$ （需要一个二维距离矩阵）
对负权边的支持	不支持。遇到带有负权值的边会产生错误结果。	支持。可以处理负权边，并且能通过矩阵对角线出现负数来检测负权环。

8.6.4. 最短路径算法比较

Dijkstra算贪心算法还是动态规划？

- 贪心选择性质
  - 局部最优策略
  - 不回溯。
- 动态规划特征
  - 最优子结构：最短路径的子路径也一定是该两点间的最短路径，这就是 DP 的心前提。
  - 状态转移方程：Dijkstra 算法中著名的“松弛操作(Relaxation)”，在数学质上就是一个标准的 DP 状态转移方程：

    `dist[v] = min(dist[v], dist[u] + weight(u, v)`
- 核心区别：状态更新的顺序和确认方式
  - 标准的动态规划（如 Bellman-Ford 算法）比较“保守”。它会在每一次迭代把所有的边都盲目地“松弛”一遍，慢慢让所有的状态收敛到最优。
  - 贪心算法（Dijkstra 算法）非常“激进且自信”。当前f值最小的那个节点，状态已经收敛，绝对不需要再更新了。
- 可以将Dijkstra 算法理解为一种使用了贪心策略来优化计算顺序的动态规划。
  - 底层支撑是DP：依赖最优子结构和状态转移（松弛）。
  - 执行驱动是贪心：依靠“无负权边”的保证，每次贪心地敲定一个节点的最终最短距离，避免了像常规 DP 那样对全图进行反复的全局盲目更新。

8.6.5 拓展思考

我们目前已经学习了不同的求解迷宫问题的方法，他们有哪些异同？

栈，队列，递归，DFS,BFS,Dijkstra,……

- 栈/递归（深度优先搜索 DFS)：不撞南墙不回头
  - 能保证“找到一条出路”，无法保证最短
- 队列(广度优先搜索 BFS)：水波纹扩散
  - 在无权迷宫中，绝对保证找到的是步数最少的最短路径
- Dijkstra（贪心算法+）
  - 在带权（非负）迷宫中，找到最短路径

8.6.5 拓展思考

我们目前已经学习了不同的求解迷宫问题的方法，他们有哪些异同？

- 普通迷宫(无权图）：**Dijkstra退化为BFS**
- 地形迷宫（有权图）：**BFS/DFS无法求解，Dijkstra求最短路径**

8.6.5 拓展示例1：地图探索的A* 算法

A*算法可以看作是“加了智能导航仪的 Dijkstra算法”

在游戏中，地图通常被划分为一个个网格（Grid）。如果使用 Dijkstra 算法，角色寻路时就像“水波纹”一样，盲目地向四面八方扩散，直到水波触碰到终点。这种盲目搜索会遍历大量无关的区域，极大地消耗 CPU 资源，导致游戏掉帧、卡顿。

A*算法的它的核心公式是：**f(n) = g(n) + h(n)**

- g(n)（实际代价）：从起点走到当前格子n已经花费的实际步数（就像Dijkstra）。
- h(n)（预估代价）：从当前格子n走到终点的预估直线距离或曼哈顿距离。这就是“导航仪”。
- f(n)(总评代价)： A*算法每次都会从已知边界中，挑选f(n)值最小（即综合看来最划算、方向最对）的格子进行探索。

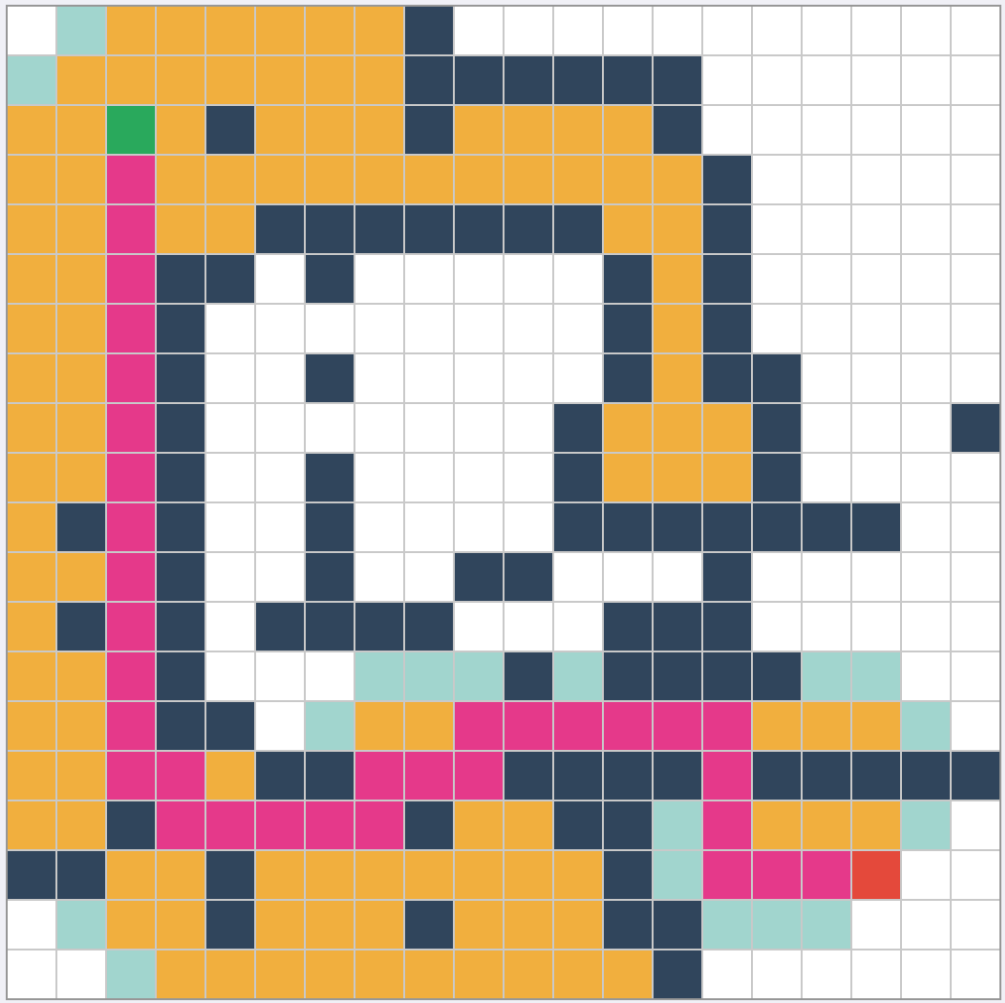
▶ 开始自动寻路　↺ 清除路径　✕ 清空障碍

操作说明： 按住鼠标左键在网格上滑动可绘制/擦除黑色障碍物（墙壁）。

核心公式： `f(n) = g(n) + h(n)` (总代价 = 已走步数代价 + 预估到终点距离)

状态：到达终点！

起点　终点　障碍　探索中 (Open)　已评估 (Closed)　最短路径



8.6.5 拓展示例2：魔方问题

问题：在一个3阶魔方中，魔方的每一个面都由 9 个方块（3×3）构成。魔方处在初始形态时，每个面的方块都是同一种颜色，魔方的六个面一共包含蓝、红、橙、绿、黄、白六种颜色。将魔方的各面随机旋转几次，如何它通过最少的旋转还原到魔方的初始形态，即各个面的方块颜色一样呢？

关键：如何将该问题转化为一个图问题？

核心思想：

将魔方的任意一个形态建模为图中的一个顶点。如果魔方的两个形态可以通过魔方一个面的一次旋转相互转化，那么就将相应的两个顶点连成一条边。这样可以得到一个魔方形态的转移图。从一个打乱的魔方还原至初始形态可以转化为在该转移图上寻找一条从打乱状态所对应顶点到初始形态所对应顶点的路径问题。而这两个顶点之间的最短路径即为最优的旋转方案

8.7 拓扑排序

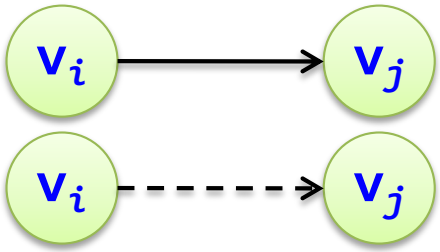
8.7.1 什么是拓扑排序

有向图

- 设G=(V，E)是一个具有n个顶点的有向图，V中顶点序列v_1、v_2、…、v_n称为一个拓扑序列，当且仅当该顶点序列满足下列条件：若<v_i，v_j>是图中的有向边或者从顶点v_i到顶点v_j有一条路径，则在序列中顶点v_i必须排在顶点v_j之前。

… v_i … v_j …



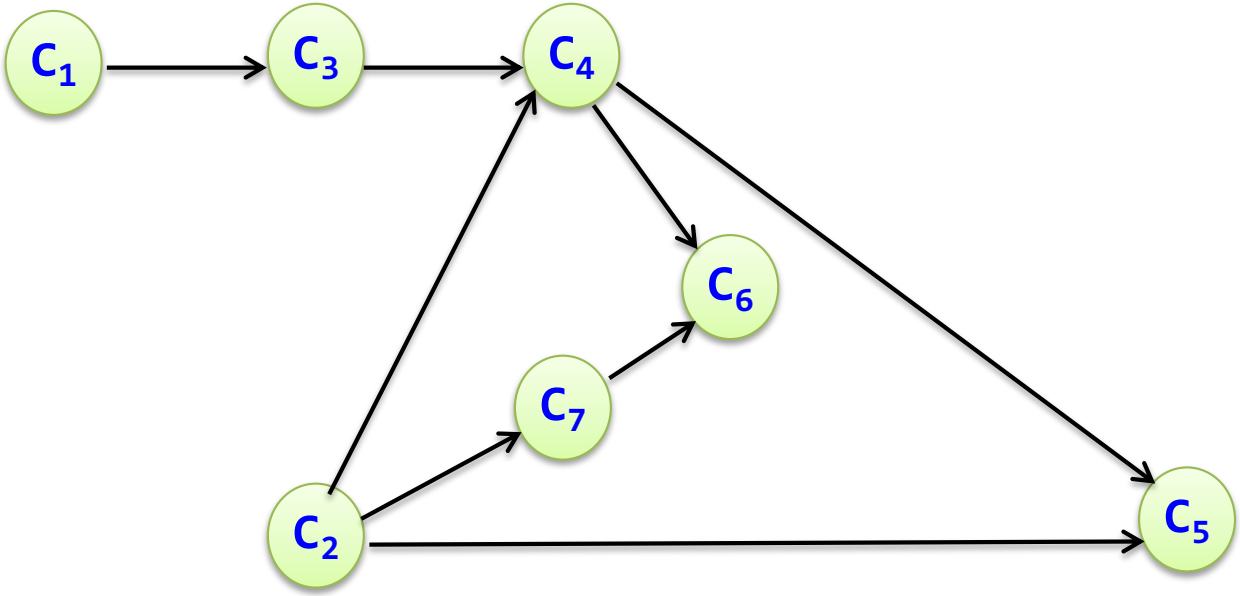


- 在一个有向图G中找一个拓扑序列的过程称为拓扑排序。

例如，计算机专业的学生必须完成一系列规定的基础课和专业课才能毕业，假设这些课程的名称与相应代号有如下关系：

课程代号	课程名称	先修课程
C_1	高等数学	无
C_2	程序设计	无
C_3	离散数学	C_1
C_4	数据结构	C_2，C_3
C_5	编译原理	C_2，C_4
C_6	操作系统	C_4，C_7
C_7	计算机组成原理	C_2

课程之间的先后关系可用有向图表示：



可以这样排课：

- C_1-C_3-C_2-C_4-C_7-C_6-C_5
- C_2-C_7-C_1-C_3-C_4-C_5-C_6

第1学期　　第2学期

拓扑排序的过程如下：

（1）从有向图中选择一个没有前驱（即入度为0）的顶点并且输出它。

（2）从图中删去该顶点，并且删去从该顶点发出的全部有向边。

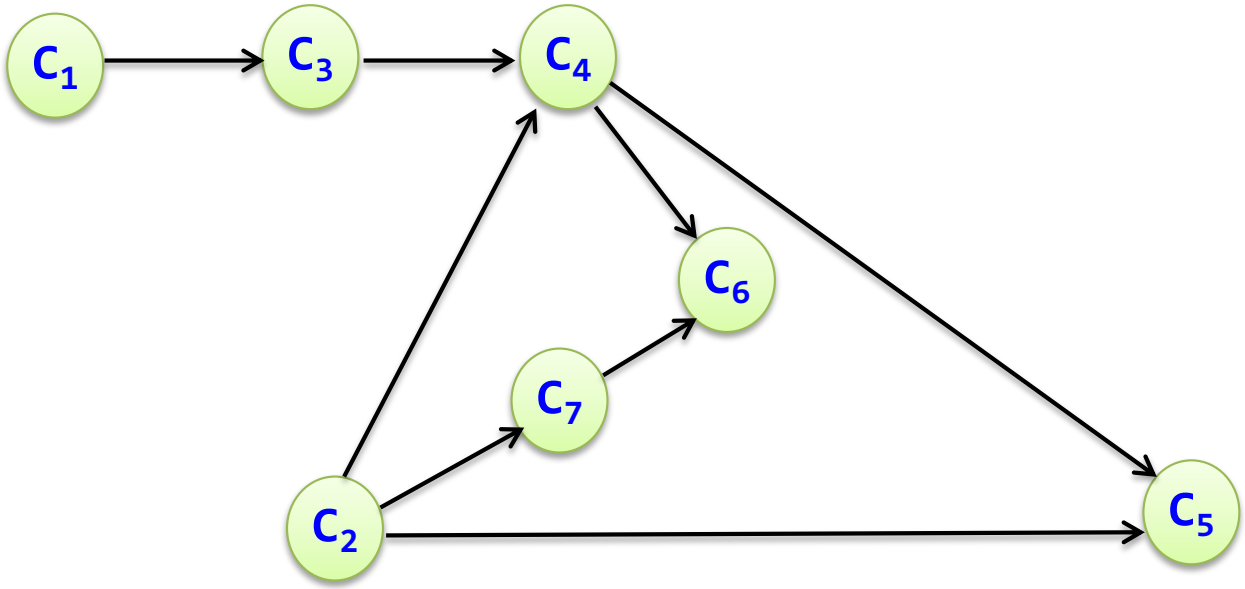
（3）重复上述两步，直到剩余的图中不再存在没有前驱的顶点为止。

拓扑排序的结果有两种：

- 一种是图中全部顶点都被输出，即得到包含全部顶点的拓扑序列，称为成功的拓扑排序。
- 另一种就是图中顶点未被全部输出，即只能得到部分顶点的拓扑序列，称为失败的拓扑排序。

↓

说明有向图中存在回路。



产生一个拓扑序列：

C_1 C_3 C_2 C_7 C_4 C_6 C_5

排序完成

8.7.2 拓扑排序算法设计

- 在设计拓扑排序算法时，假设给定的有向图采用邻接表作为存储结构。
- 需要考虑顶点的入度，为此设计一个ind数组，ind[i]存放顶点i的入度，先通过邻接表G求出ind。
- 拓扑排序是设计要点如下：

  - 在某个时刻，可以有多个入度为0的顶点，为此设置一个栈st，以存放多个入度为0的顶点，栈中的顶点的都是入度为0的顶点。
  - 出栈顶点i时，将顶点i输出，同时删去该顶点的所有出边，实际上没有必要真的删去这些出边，只需要将顶点i的所有出边邻接点的入度减1就可以了。

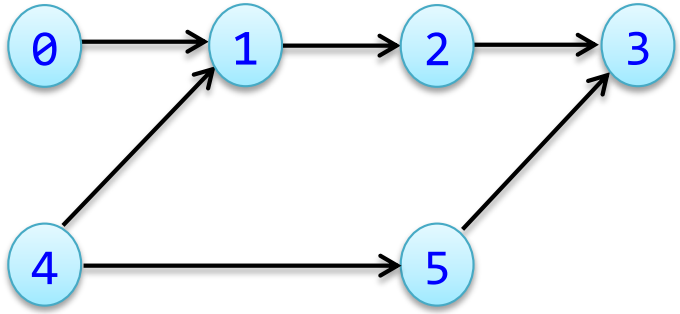
```
void TopSort(AdjGraph& G)                    //拓扑排序
{  stack<int> st;                            //定义一个栈
   int ind[MAXV];                            //记录每个顶点的入度
   memset(ind,0,sizeof(ind));
   ArcNode* p;
   for (int i=0;i<G.n;i++)                   //求所有顶点的入度
   {  p=G.adjlist[i].firstarc;
      while (p!=NULL)                        //处理顶点i的所有出边
      {  int w=p->adjvex;                    //存在有向边<i,w>
         ind[w]++;                           //顶点w的入度增1
         p=p->nextarc;
      }
   }

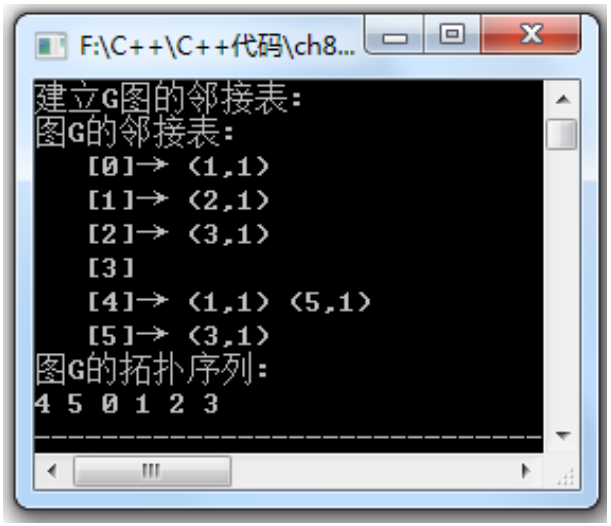
   for (int i=0;i<G.n;i++)                   //将所有入度为0的顶点进栈
      if (ind[i]==0)
         st.push(i);
```

```
    while (!st.empty())                  //栈不为空时循环
    {  int i=st.top(); st.pop();         //出栈一个顶点i
       printf("%d ",i);                  //输出拓扑序列中的一个顶点i

       p=G.adjlist[i].firstarc;          //找顶点i的第一个邻接点
       while (p!=NULL)
       {  int w=p->adjvex;               //邻接点为w
          ind[w]--;                      //顶点w的入度减1
          if (ind[w]==0)                 //入度为0的邻接点w进栈
             st.push(w);
          p=p->nextarc;                  //找下一个邻接点
       }
    }
}
```

时间复杂度为$O(n+e)$。


0
1
2
3
4
5


F:\C++\C++代码\ch8...
建立G图的邻接表:
图G的邻接表:
[0]→ (1,1)
[1]→ (2,1)
[2]→ (3,1)
[3]
[4]→ (1,1) (5,1)
[5]→ (3,1)
图G的拓扑序列:
4 5 0 1 2 3

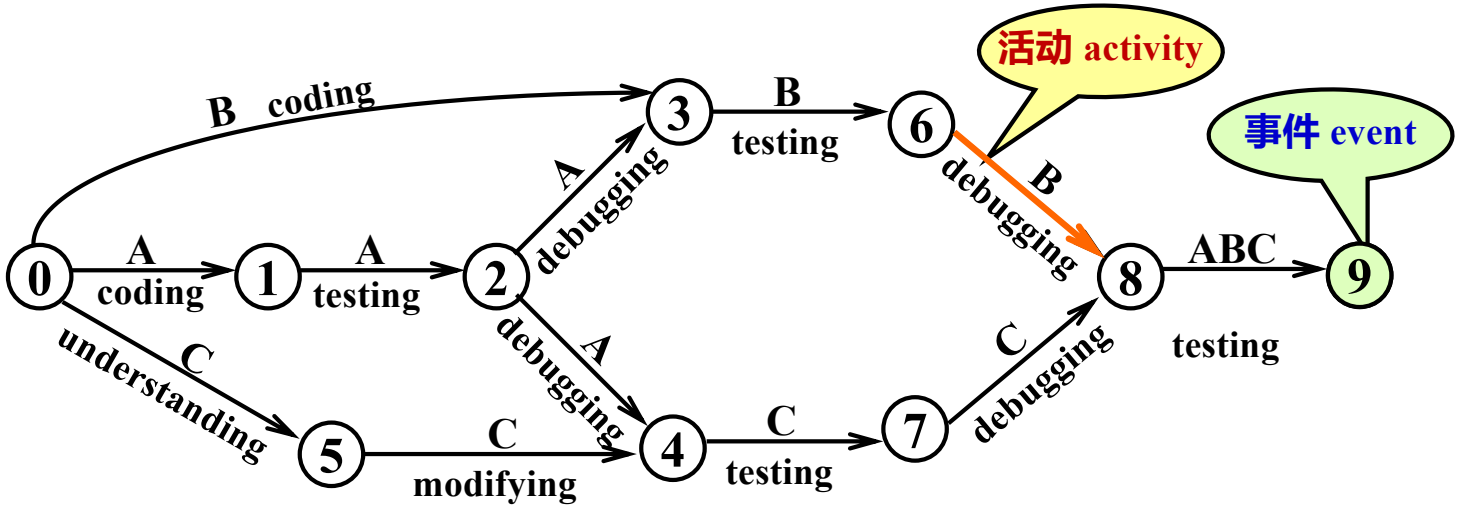
说明

拓扑排序中栈仅仅用于存放所有入度为0的顶点，不必考虑先后顺序，可以用队列替代栈。

8.8 AOE网与关键路径

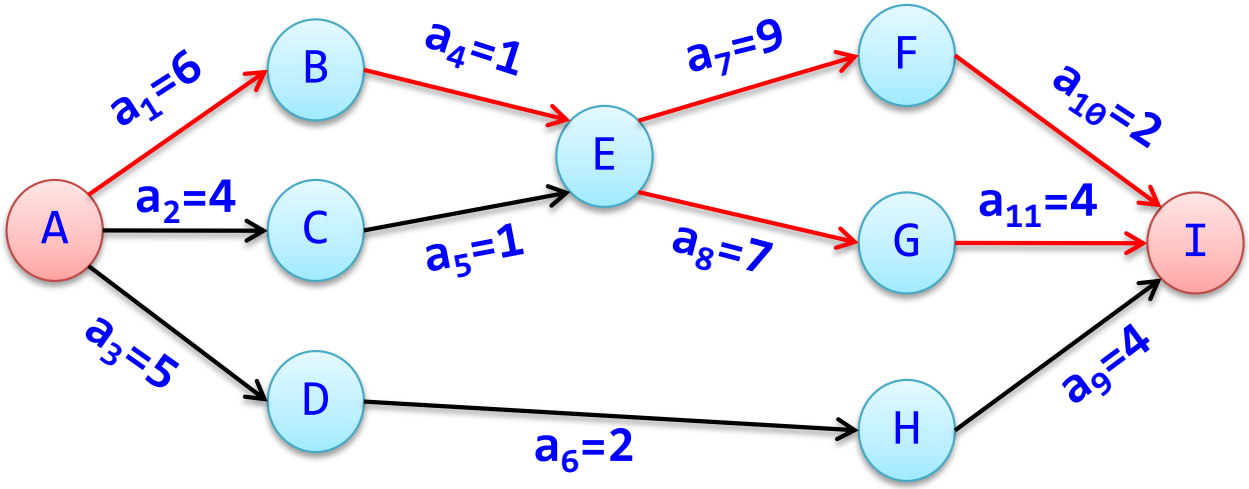


- 开发三个模块A、B、C。
- A为公用模块，B、C的测试须等A的调试完成后进行。
- A的编码需6天，测试8天，调试6天。
- B的编码需7天，测试8天，调试6天。
- C利用已有的模块，须先理解原模块8天，再修改8天，测试9天，调试7天。
- 最后三模块集成测试需5天完成。



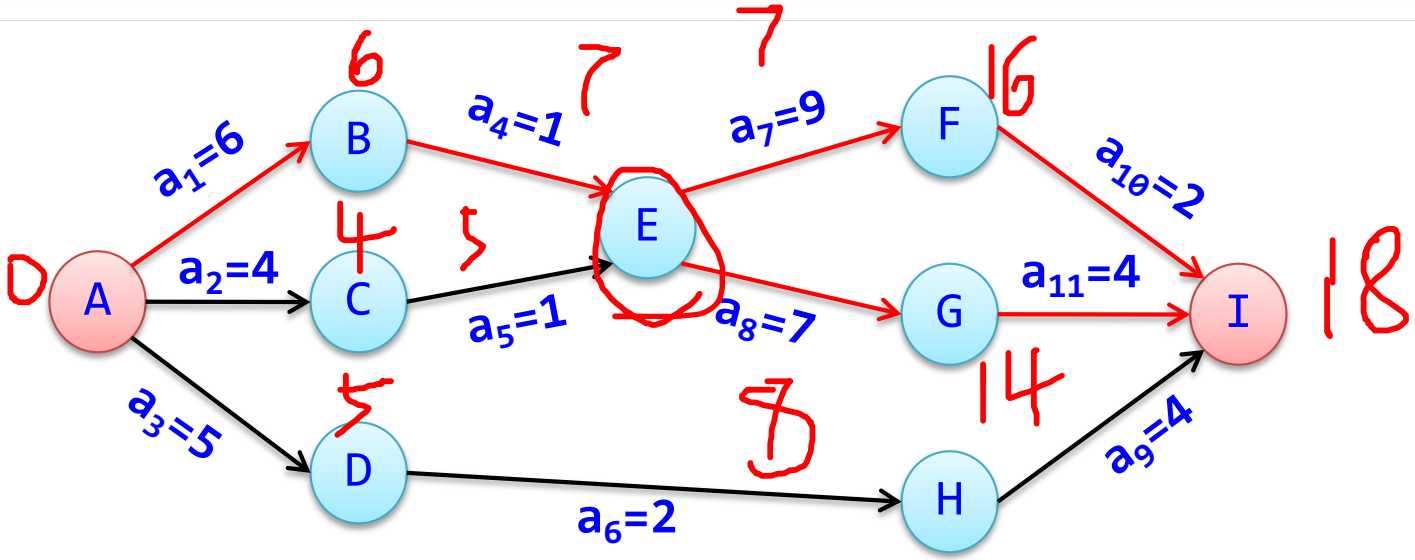
8.8 AOE网与关键路径

- 若用一个带权有向无环图（DAG）描述工程的预计进度，以顶点表示事件，有向边表示活动，边e的权c(e)表示完成活动e所需的时间（比如天数），或者说活动e持续时间 ⇨ AOE网。
- 通常AOE网中只有一个入度为0的顶点，称为源点，和一个出度为0的顶点，称为汇点。



- 源点：A，　汇点：I

- 在AOE网中，从源点到汇点的所有路径中，具有最大路径长度的路径称为关键路径。完成整个工程的最短时间就是网中关键路径的长度。
- 关键路径上的活动称为关键活动，或者说关键路径是由关键活动构成的。
- 只要找出AOE网中的全部关键活动，也就找到了全部关键路径了。



- 关键路径：A → B → E → F → I，　A → B → E → G → I
- 关键活动：$a_1, a_4, a_7, a_8, a_{10}, a_{11}$

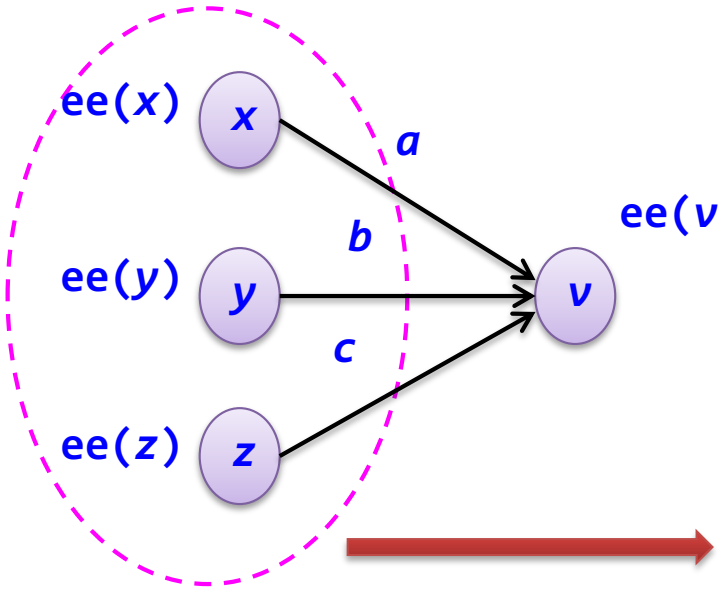
求关键路径的过程

（1）事件的最早开始和最迟开始时间

事件v的最早开始时间：规定源点事件的最早开始时间为0。定义图中任一事件v的最早开始时间ee(v)等于x、y、z到v所有路径长度的最大值：

$$ee(v)=0 \quad \text{当v为源点时}$$

$$ee(v)=MAX\{ee(x)+a,\ ee(y)+b,\ ee(z)+c\} \quad \text{否则}$$

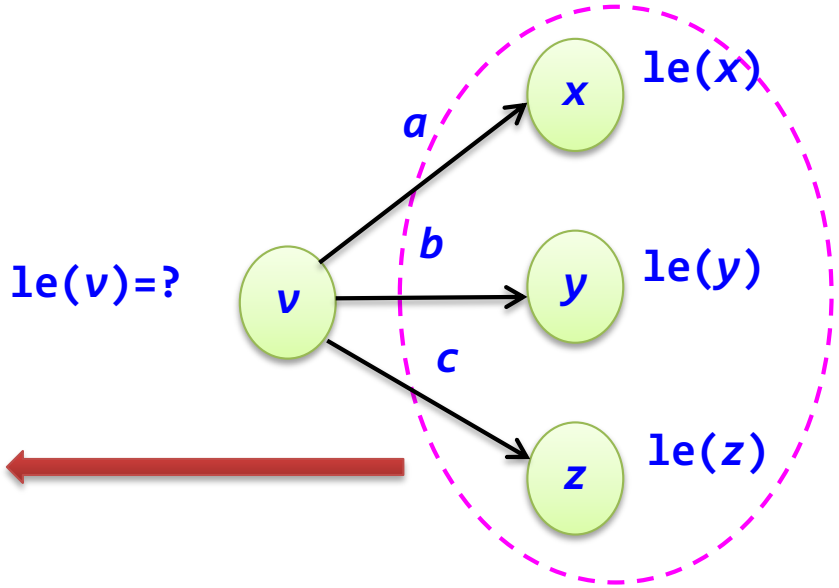


从左向右推进计算

这是为什么源点要唯一！

事件v的最迟开始时间：定义在不影响整个工程进度的前提下，事件v必须发生的时间称为v的最迟开始时间le(v)应等于ee(y)与v到汇点的最长路径长度之差：

$$
\begin{cases} le(v)=ee(v) & \text{当}v\text{为汇点时} \\ le(v)=\min\{le(x)-a,\ le(y)-b,\ le(z)-c\} & \text{否则} \end{cases}
$$



从右向左推进计算

这是为什么汇点要唯一！

（2）活动的最早开始时间和最迟开始时间



活动a的最早开始时间$e(a)$指该活动起点x事件的最早开始时间，即：

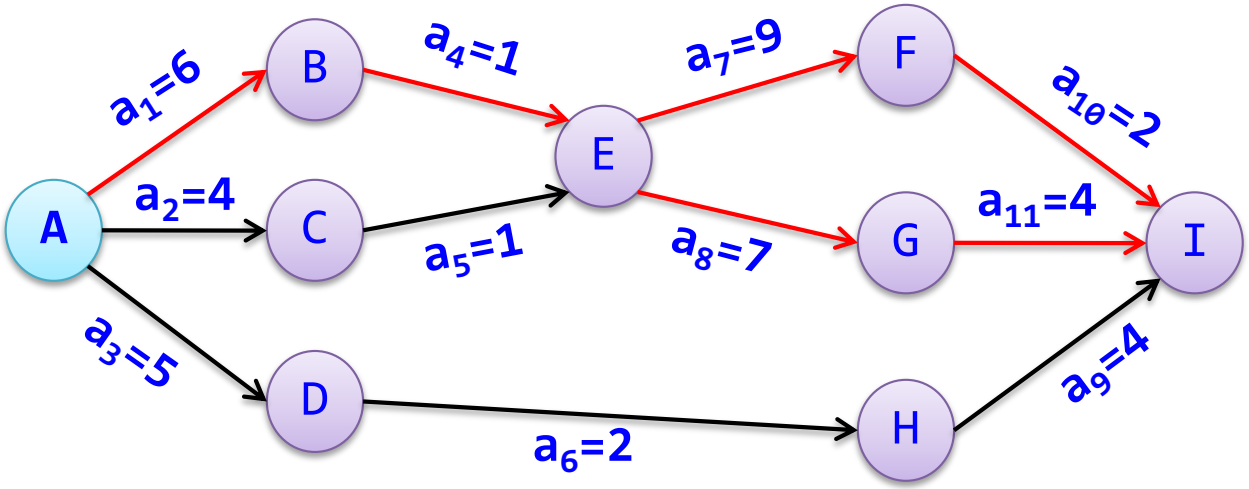
$$e(a)=ee(x)$$

活动a的最迟开始时间$l(a)$指终点y事件的最迟开始时间与该活动所需时间之差，即：

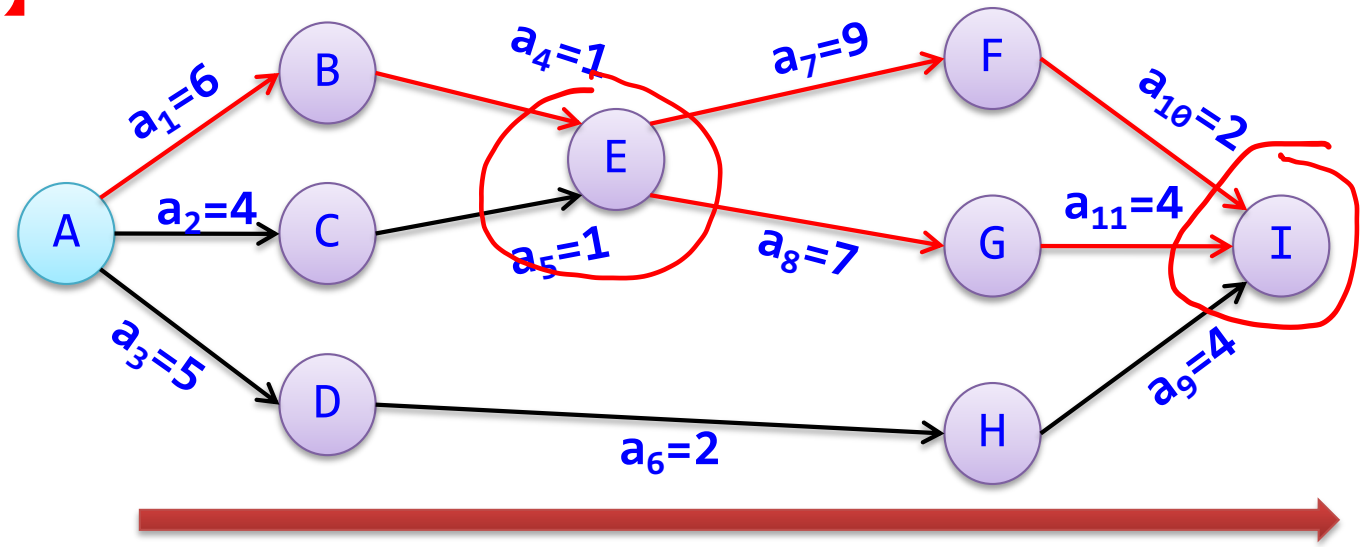
$$l(a)=le(y)-c$$

（3）求关键活动

对于每个活动a，求出$d(a)=L(a)-e(a)$，若$d(a)$为0，则称活动a为关键活动。 对关键活动来说，不存在富余时间。



【例8.13】



先进行拓扑排序，假设拓扑序列为：ABCDEFGHI

计算各事件的ee(v)如下：

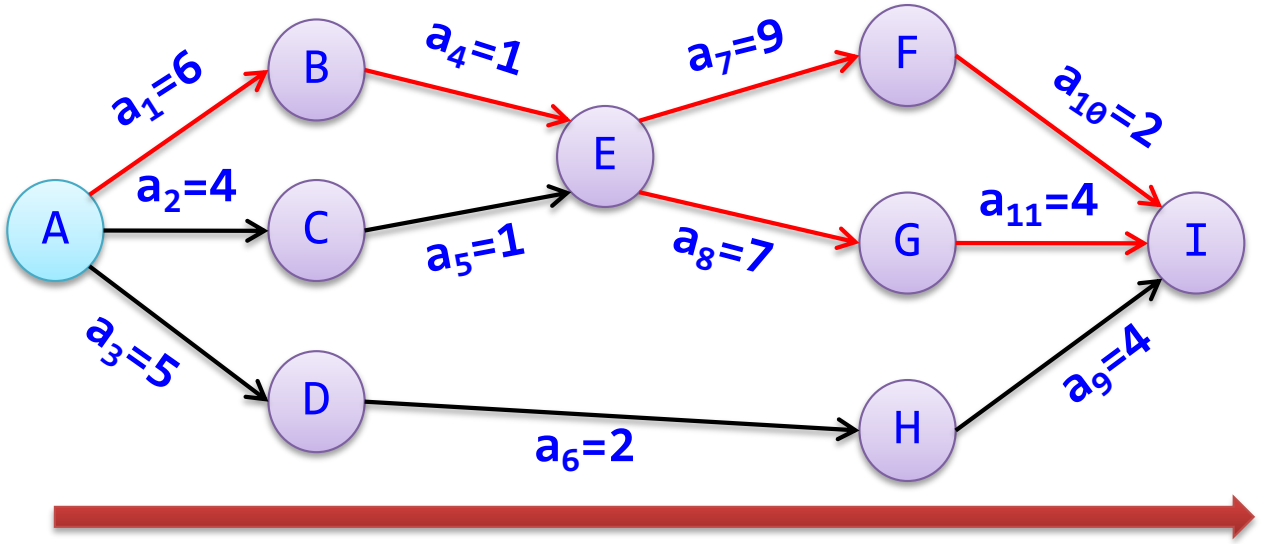
ee(A)=0

ee(B)=ee(A)+c(a_1)=6

ee(C)=ee(A)+c(a_2)=4

ee(D)=ee(A)+c(a_3)=5

ee(E)=MAX(ee(B)+c(a_4), ee(C)+c(a_5)}=MAX{7, 5}=7



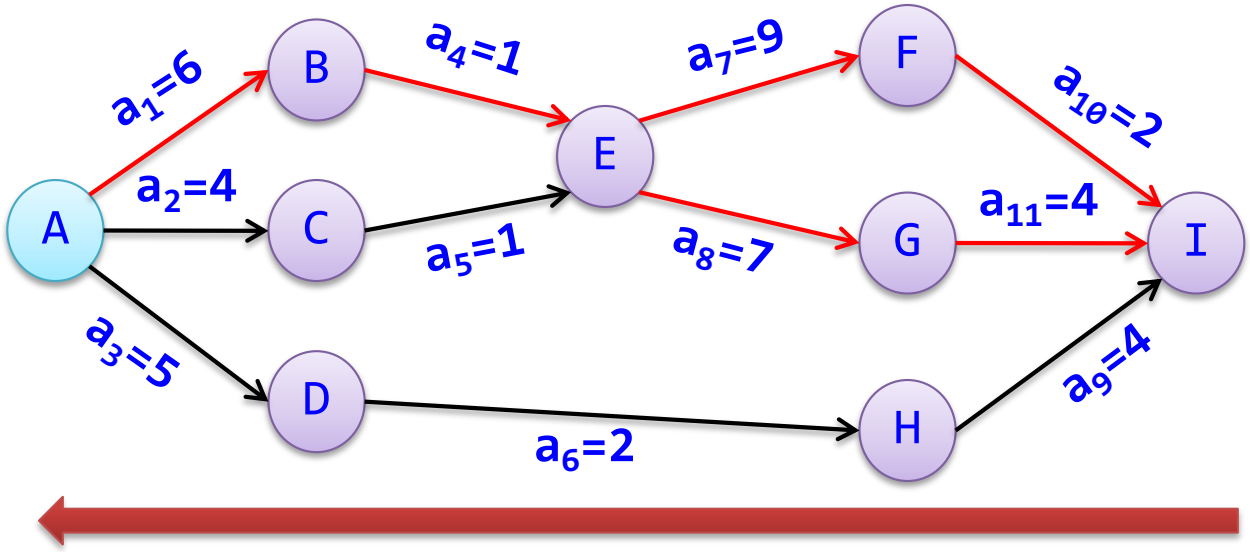
ee(F)=ee(E)+c(a_7)=16

ee(G)=ee(E)+c(a_8)=14

ee(H)=ee(D)+c(a_6)=7

ee(I)=MAX{ee(F)+c(a_{10}), ee(G)+c(a_{11}), ee(H)+c(a_9)}

=MAX(18, 18, 11}=18



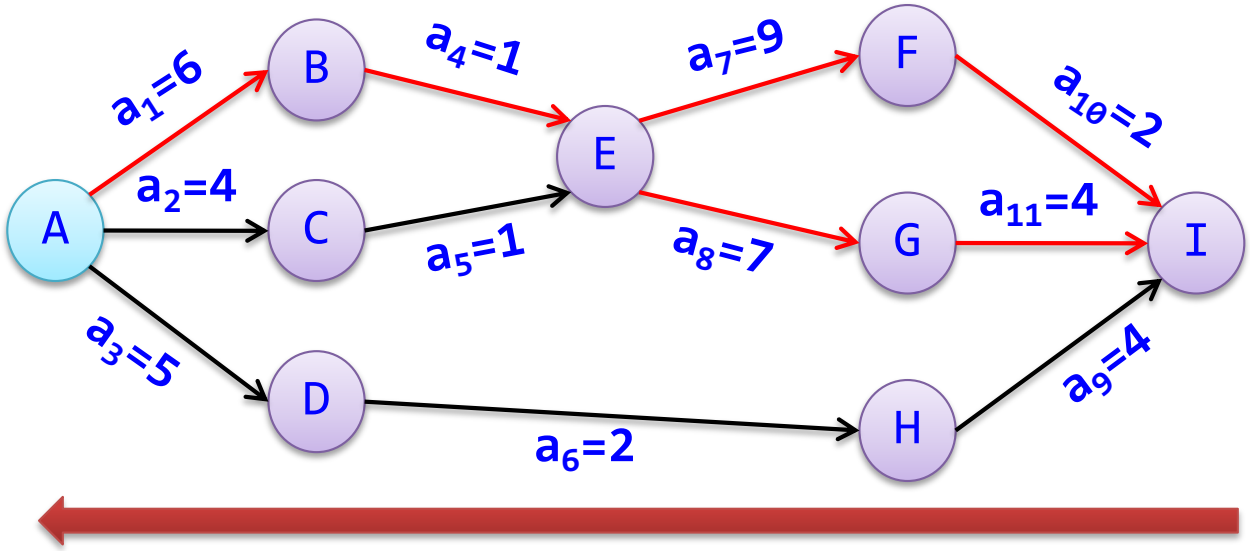
拓扑序列为ABCDEFGHI，按拓扑逆序IHGFEDCBA计算各事件的le(v)如下：

le(I)=ee(I)=18

le(H)=le(I)-c(a_9)=14

le(G)=le(I)-c(a_{11})=14

le(F)=le(I)-c(a_{10})=16



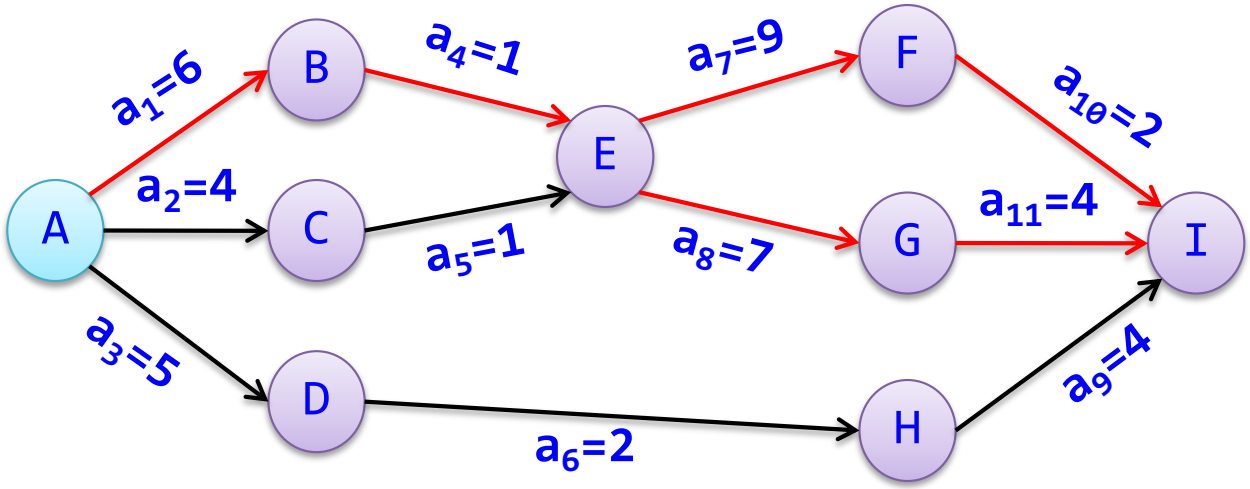
$le(E)=MIN(le(F)-c(a_7), le(G)-c(a_8)\}=\{7, 7\}=7$

$le(D)=le(H)-c(a_6)=12$

$le(C)=le(E)-c(a_5)=6$

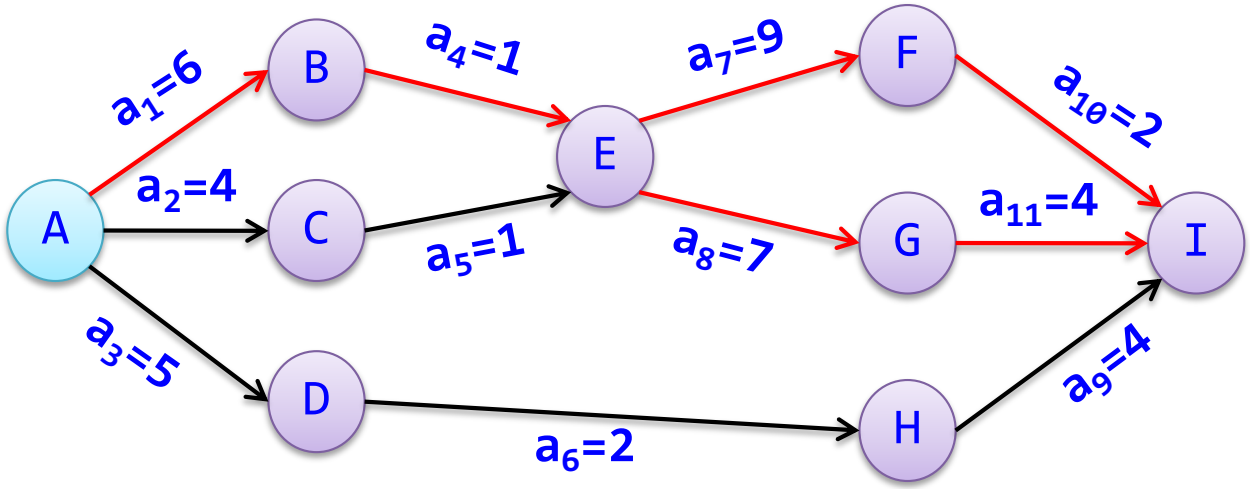
$le(B)=le(E)-c(a_4)=6$

$le(A)=MIN(le(B)-c(a_1), le(C)-c(a_2), le(D)-c(a_3)\}=\{0, 2, 7\}=0$

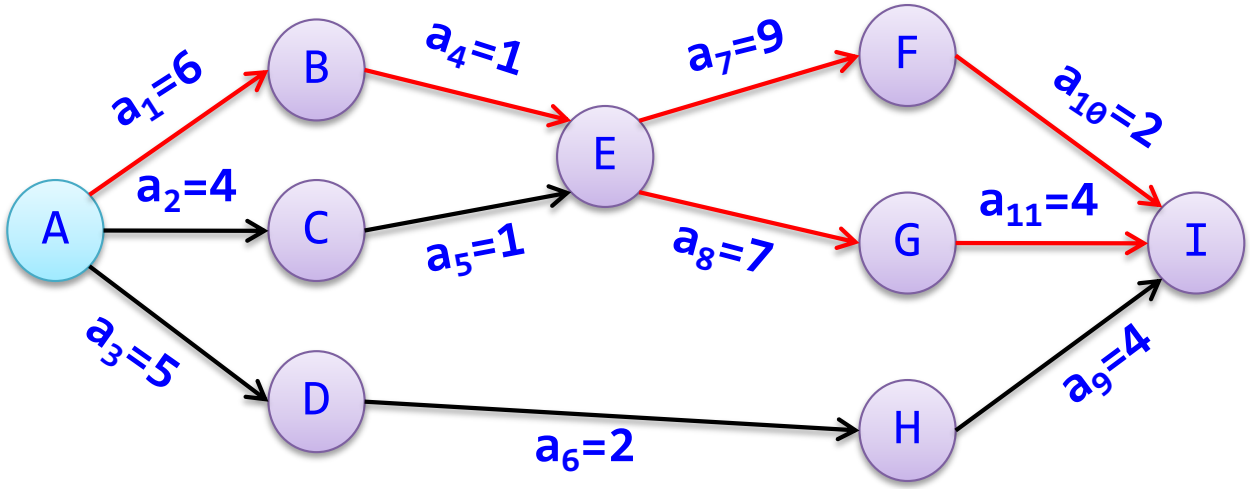


计算各活动的e(a)、l(a)和d(a)如下：

活动a_1:	$e(a_1)=ee(A)=0$,	$l(a_1)=le(B)-6=0$,	$d(a_1)=0$
活动a_2:	$e(a_2)=ee(A)=0$,	$l(a_2)=le(C)-4=2$,	$d(a_2)=2$
活动a_3:	$e(a_3)=ee(A)=0$,	$l(a_3)=le(D)-5=7$,	$d(a_3)=7$
活动a_4:	$e(a_4)=ee(B)=6$,	$l(a_4)=le(E)-1=6$,	$d(a_4)=0$
活动a_5:	$e(a_5)=ee(C)=4$,	$l(a_5)=le(E)-1=6$,	$d(a_5)=2$

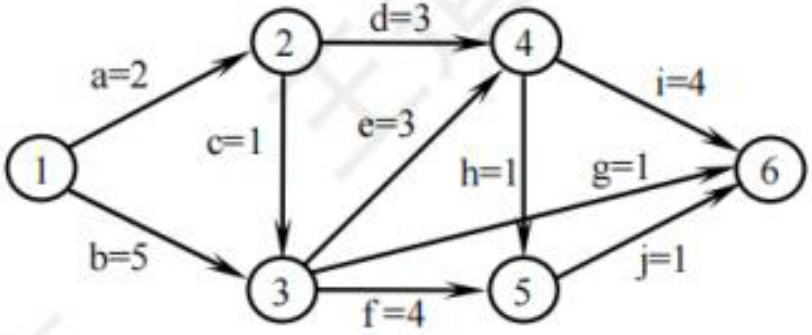


活动a_6: $e(a_6)=ee(D)=5$,	$l(a_6)=le(H)-2=12$,	$d(a_6)=7$
活动a_7: $e(a_7)=ee(E)=7$,	$l(a_7)=le(F)-9=7$,	$d(a_7)=0$
活动a_8: $e(a_8)=ee(E)=7$,	$l(a_8)=le(G)-7=7$,	$d(a_8)=0$
活动a_9: $e(a_9)=ee(H)=7$,	$l(a_9)=le(I)-4=14$,	$d(a_9)=7$
活动a_{10}: $e(a_{10})=ee(F)=16$,	$l(a_{10})=le(I)-2=16$,	$d(a_{10})=0$
活动a_{11}: $e(a_{11})=ee(G)=14$,	$l(a_{11})=le(I)-4=14$,	$d(a_{11})=0$



由此可知，关键活动有a_{11}、a_{10}、a_8、a_7、a_4、a_1，因此关键路径有两条：A—B—E—F—I和A—B—E—G—I。

【2022 统考真题】下图是一个有 10 个活动的 AOE 网，时间余量最大的活动是（ ）。



A. c　　B. g　　C. h　　D. j

45. B

在 AOE 网中，活动的时间余量＝结束顶点的最迟开始时间－开始顶点的最早开始时间－该活动的持续时间。根据关键路径算法得到下表：

结点编号	1	2	3	4	5	6
最早开始时间 $v_e(i)$	0	2	5	8	9	12
最迟开始时间 $v_l(i)$	0	4	5	8	11	12

c 的时间余量＝$v_l(3)-v_e(2)-1=5-2-1=2$，g 的时间余量＝$v_l(6)-v_e(3)-1=12-5-1=6$，h 的时间余量＝$v_l(5)-v_e(4)-1=11-8-1=2$，j 的时间余量＝$v_l(6)-v_e(5)-1=12-9-1=2$。



思考题

在一个AOE网中，缩短任一关键活动的时间，是否会缩短整个工程的时间？

- 缩短某一活动的时间，整个工期一定会缩短 ✗
- 缩短某一关键活动的时间，整个工期一定会缩短 ✗
- 缩短所有关键路径共享的关键活动的时间，整个工期会缩短 ✓
- 缩短某一共享关键活动时间为d(d>0)天，整个工期一定会缩短d天 ✗

